Vibe Coding VCBD dengan Claude AI

Vibe Coding VCBD dengan Claude AI

Panduan Praktis Menyusun Blueprint, Membangun, dan Mengaudit Aplikasi dengan VCBD Suite

Syams Ideris

Edisi Pertama · 2026
Berdasarkan VCBD Suite v2.6

Vibe Coding VCBD dengan Claude AI
Panduan Praktis Menyusun Blueprint, Membangun, dan Mengaudit Aplikasi dengan VCBD Suite

Penulis: Syams Ideris
Edisi Pertama, Oktober 2026
Ukuran: 6 × 9 inci (15,24 × 22,86 cm)

Hak Cipta © 2026 Syams Ideris

Buku ini adalah dokumen bebas: Anda boleh menyebarluaskan dan/atau mengubahnya di bawah ketentuan **GNU General Public License versi 3** sebagaimana diterbitkan oleh Free Software Foundation, atau (sesuai pilihan Anda) versi yang lebih baru. Buku ini disebarkan dengan harapan dapat bermanfaat, tetapi TANPA JAMINAN APA PUN; bahkan tanpa jaminan tersirat atas KELAYAKAN DIPERDAGANGKAN atau KESESUAIAN UNTUK TUJUAN TERTENTU. Teks lengkap lisensi tercantum di Lampiran F, dan juga tersedia di `https://www.gnu.org/licenses/gpl-3.0.html`.

Perangkat lunak VCBD Suite yang dibahas dalam buku ini dilisensikan secara terpisah di bawah GNU General Public License versi 2 (GPL-2.0), sebagaimana tercantum pada berkas `LICENSE` di tiap folder skill. Kode sumbernya tersedia di `https://github.com/Syamsuddin/VCBD`.

Claude dan Claude Code adalah merek dagang Anthropic. Penyebutan merek dalam buku ini hanya untuk keperluan identifikasi dan tidak menyiratkan dukungan atau afiliasi.

Seluruh keluaran terminal dalam buku ini diperoleh dari menjalankan skrip VCBD Suite v2.6 pada proyek demo PRESQU dan SI-CUTI, kemudian dipangkas agar fokus. Data pada proyek demo adalah data rekaan; nama, NIP, dan kode yang tampak tidak merujuk pada orang atau instansi nyata.

ISBN: —
(belum diterbitkan dengan ISBN)

Huruf isi: Source Serif 4 · Judul: Inter · Kode: JetBrains Mono

Kata Pengantar

Beberapa tahun terakhir, cara kita membangun perangkat lunak berubah lebih cepat daripada kebiasaan kita mengikutinya. Hari ini, seorang analis di kantor pemerintahan, seorang guru, atau seorang pengembang tunggal bisa mendeskripsikan aplikasi dalam bahasa sehari-hari, lalu melihat agen AI menuliskan kodenya. Orang menyebutnya *vibe coding*. Rasanya seperti sihir, sampai aplikasinya harus dipelihara.

Buku ini lahir dari pengalaman yang cukup sering terulang: aplikasi yang dibangun dengan cepat ternyata membangun hal yang sedikit keliru, dan dokumen yang mestinya menjadi pegangan justru saling bertentangan. Agen AI tidak bersalah dalam hal ini. Ia mengerjakan persis apa yang ia pahami. Yang kurang adalah gambar kerja yang jelas, konsisten, dan bisa diperiksa.

VCBD Suite adalah upaya menjawab kekurangan itu. Lima skill Claude Code di dalamnya bekerja sebagai satu rantai: menggali kebutuhan sebelum menulis, menyusun blueprint yang tidak saling bertentangan, membangun aplikasi slice demi slice, mengaudit hasilnya terhadap blueprint, dan membedah aplikasi lama yang tidak punya dokumen. Prinsipnya sederhana, walau tidak selalu mudah: **gali dan konfirmasi dulu, baru tulis**; **satu fakta, satu rumah**; dan **selesai berarti diuji, bukan diakui**.

Buku ini ditulis sebagai tutorial. Setiap bab skill memakai proyek demo yang benar-benar dijalankan, dan setiap tangkapan layar terminal adalah keluaran asli skrip, bukan rekaan. Saya sengaja mempertahankan bagian yang "tidak mulus": gerbang yang

menolak, temuan palsu yang harus ditolak, tes regresi yang ternyata belum menangkap celah, dan sebuah bug sungguhan yang muncul saat buku ini disusun. Justru di bagian-bagian itulah cara kerja VCBD paling terlihat.

Buku ini ditujukan bagi pengembang yang memakai Claude Code, pengelola aplikasi di instansi yang mewarisi sistem tanpa dokumentasi, dan siapa pun yang ingin agen AI-nya bekerja dengan disiplin. Pengetahuan dasar tentang terminal, Git, dan satu bahasa pemrograman akan membantu, tetapi tidak wajib.

Buku ini dirilis di bawah GNU General Public License versi 3. Silakan salin, ubah, dan bagikan. Bila Anda menemukan kekeliruan, perbaikan Anda akan sangat berharga bagi pembaca berikutnya.

Terima kasih kepada semua yang telah mencoba versi-versi awal VCBD dan dengan sabar melaporkan di mana ia tersandung. Setiap laporan itu kini menjadi satu baris aturan, satu cek validator, atau satu anti-pattern yang dicegah.

Oktober 2026
Syams Ideris

Daftar Isi

Kata Pengantar vii

Daftar Gambar xiii

Bab 1 Mengenal Vibe Coding dan VCBD 1

1.1 Vibe coding: cepat, menyenangkan, dan licin 1
1.2 Jawaban VCBD dalam dua kalimat 2
1.3 Lima skill, satu siklus hidup 3
1.4 Empat konvensi bersama 4
1.5 Kapan VCBD cocok, dan kapan tidak 4
1.6 Cara membaca buku ini 5

Bab 2 Menyiapkan Bengkel Kerja 7

2.1 Kebutuhan sistem 7
2.2 Memasang di Claude Code 8
2.3 Mengapa harus bersebelahan? 9
2.4 Menyalin skrip ke proyek 9
2.5 Memasang di claude.ai 10
2.6 Bila salinan skill ganda 11
2.7 Memeriksa instalasi 11
2.8 Memperbarui dan mencopot 12

Bab 3 vcbd: Menyusun Blueprint 13

3.1 Empat Hukum vcbd 13
3.2 Anatomi paket blueprint 14
3.3 Fase 0: Triase 15
3.4 Fase 1: Wawancara penggalian 16
3.5 Fase 2: Konfirmasi (gerbang keras) 19
3.6 Fase 3: Generasi 20
3.7 Fase 4: Serah terima 26
3.8 Mode Pembaruan 27

3.9 Mode SPLIT: backend dan frontend terpisah 27
3.10 Anti-pattern yang dicegah aktif 28

Bab 4 coding-vcbd: Membangun Slice demi Slice 31
4.1 Dua kegagalan yang dicegah 31
4.2 Empat Hukum Eksekusi 32
4.3 Peta tujuh fase 32
4.4 Fase A: Muat Tier-0 dan kunci sesi 32
4.5 Fase B: Ambil satu slice dan rencanakan 33
4.6 Fase C: Implementasi berlapis 34
4.7 Fase D: Verifikasi 36
4.8 Fase E: Gerbang selesai fitur 37
4.9 Fase F: Tutup slice 39
4.10 Fase G: Kapan aplikasi boleh disebut selesai 40
4.11 Diet token: aturan operasional 41

Bab 5 review-vcbd: Audit, Uji, Lacak, Tambal 44
5.1 Empat Hukum Audit 44
5.2 Lima mode 45
5.3 Skenario: kode yang masuk lewat pintu samping 45
5.4 Mode PINDAI 46
5.5 Mode REVIEW: verifikasi temuan 47
5.6 Mode UJI 49
5.7 Mode LACAK: sebuah bug sungguhan 49
5.8 Mode TAMBAL: menutup TEM-001 51
5.9 Laporan dan syarat sehat 53
5.10 Anti-pattern audit 53

Bab 6 reverse-vcbd: Membedah Kode Warisan 55
6.1 Mengapa butuh alat khusus? 55
6.2 Lima Hukum 56
6.3 Tiga mode 56
6.4 Proyek contoh: SI-CUTI 57
6.5 Fase 0: Triase dan izin 57
6.6 Fase 1 dan 2: Pindai statis dan pindai data 57
6.7 Kode vs data: siklus status pengajuan 59
6.8 Fase 3 dan 4: Wawancara kilat dan gerbang Ringkasan Temuan 60

6.9 Fase 5: Generasi dan gerbang mesin 61
6.10 Derajat bukti 61
6.11 Fase 6: Serah terima 62

Bab 7 reverse-web-vcbd: Membedah Kotak Hitam 63
7.1 Lima Hukum 63
7.2 Menangkap HAR per peran 64
7.3 Bedah dan rakit dengan satu perintah 65
7.4 Membaca laporan cakupan 66
7.5 Wawancara kilat dan serah ke vcbd 67
7.6 Uji regresi skill 68

Bab 8 Merangkai Suite dalam Kerja Sehari-hari 71
8.1 Tiga jalur perjalanan 71
8.2 Ritme satu sesi 71
8.3 Meteran token 72
8.4 Bekerja dalam tim 73
8.5 Masalah umum dan jalan keluarnya 73
8.6 Berkas state yang lahir di proyek 74
8.7 Sepuluh kebiasaan baik 75

Penutup 77
Yang sudah kita lihat bekerja 77
Yang tidak dijanjikan 78
Langkah Anda berikutnya 78

Lampiran A Peta 28 Dokumen dan Pemilik Fakta 80

Lampiran B Rujukan Cepat Perintah Skrip 82

Lampiran C Label Bukti dan Status 84

Lampiran D Glosarium 86

Lampiran E Kode Sumber Inti Demo PRESQU 88

Lampiran F GNU General Public License versi 3 90

Daftar Gambar

2.1 Memasang kelima skill ke ~/.claude/skills dan memeriksa prasyaratnya. 8

2.2 Mengemas lima zip skill untuk diunggah ke claude.ai. 10

3.1 Kerangka contoh-preview.html bawaan skill, dirender apa adanya. Hanya blok token :root dan teks {ISI:} yang diganti saat preview dibuat. 18

3.2 Potongan _MANIFEST.json PRESQU (dipadatkan). 20

3.3 Menjalankan scaffold.py pada proyek PRESQU. 21

3.4 validate.sh menolak paket yang masih berupa kerangka. 21

3.5 07_DATA_MODEL.md: satu-satunya rumah skema di seluruh paket. 22

3.6 Blok fitur F1 di 23_ACCEPTANCE_CRITERIA.md. 23

3.7 26_UI_CONVENTIONS.md: token beku, empat state, larangan. 23

3.8 CLAUDE.md PRESQU setelah slot [ISI:] diisi. 24

3.9 Rute konteks di INDEX.md: jenis task menentukan dokumen yang dimuat. 25

3.10 Paket PRESQU lulus sebelas cek konsistensi. 26

4.1 Membuka sesi: init, perintah, segel, dan fitur-mulai. 33

4.2 Tiga step pertama slice F1 beserta counter-nya. 35

4.3 Empat state wajib halaman /scan: kosong (kiri atas), memuat (kanan atas), gagal (kiri bawah), sukses (kanan bawah). 36

4.4 Empat tes alur kritikal F1 lulus. 36

4.5 Gerbang menolak slice F1 versi pertama. 37

4.6 Gerbang lulus setelah dua perbaikan. 38

4.7 Menutup slice, melihat rekap, dan menerbitkan kontrak serah-terima. 40

4.8 Potongan docs/_SERAH_BUILD.json. 41

5.1 Pemindaian penuh PRESQU dalam mode audit terarah. 46

5.2 Daftar temuan beserta buktinya. 47

5.3 Menolak false positive dan menyetujui temuan yang akan ditambal. 48

5.4 Uji asap empat rute PRESQU yang sedang berjalan. 49

5.5 Reproduksi: pindaian gagal yang diikuti pindaian sah menghasilkan galat. 50
5.6 Tes regresi merah, tambalan enam baris, lalu suite penuh hijau. 51
5.7 Siklus TAMBAL lengkap untuk TEM-001. 52
5.8 Laporan temuan setelah satu tambalan dan satu penolakan. 53
6.1 Pemindaian statis lalu pemindaian data SI-CUTI. 57
6.2 Laporan cakupan SI-CUTI dengan angka apa adanya. 58
6.3 Mesin status di dosir: gabungan bukti kode dan bukti data. 59
6.4 Dosir dirakit menjadi _MANIFEST.json. 61
7.1 Satu perintah membedah tiga bahan dan mencetak ringkasan ≤15 baris. 65
7.2 Matriks akses per modul, transisi status, dan alur lintas peran. 66
7.3 Uji regresi reverse-web-vcbd: 32 dari 32 cek lulus. 68
8.1 Buku besar token PRESQU sejak genesis. 72

BAB 1

Mengenal Vibe Coding dan VCBD

Bab ini menjelaskan masalah yang ingin dipecahkan VCBD, lima skill yang membentuk suite-nya, dan empat konvensi yang membuat kelimanya bisa saling menyambung.

Setelah membaca bab ini, Anda dapat:

- menjelaskan dua kegagalan termahal dalam *vibe coding*;
- memilih skill VCBD yang tepat untuk titik awal proyek Anda;
- memahami empat konvensi bersama yang dipakai di seluruh buku.

1.1 Vibe coding: cepat, menyenangkan, dan licin

Vibe coding adalah cara membangun perangkat lunak dengan mendeskripsikan maksud kepada agen AI, lalu membiarkan agen itu menulis sebagian besar kodenya. Dengan Claude Code, Anda cukup mengetik "buatkan halaman pindai kehadiran", dan beberapa menit kemudian berkasnya sudah ada, tesnya jalan, dan aplikasinya bisa dibuka di peramban.

Kecepatan itu nyata. Tetapi kecepatan juga memperbesar akibat dari kesalahan arah. Agen AI tidak lelah dan tidak ragu: ia akan membangun persis apa yang ia pahami, dengan nada sangat yakin, sekalipun pemahamannya keliru. Pengalaman di lapangan

menunjukkan bahwa kesalahan termahal jarang berupa *kode yang salah*. Kode yang salah murah diperbaiki. Yang mahal adalah dua hal berikut.

1

Membangun hal yang salah

Kebutuhan tidak pernah digali. Agen mengisi kekosongan dengan tebakan yang masuk akal, dan tebakan itu baru ketahuan salah setelah puluhan berkas berdiri di atasnya.

2

Dokumen yang saling bertentangan

Satu berkas menyebut MySQL, berkas lain menyebut PostgreSQL. Agen menerima dua sinyal yang bertabrakan, lalu memilih salah satu secara acak. Inilah *document drift*.

Kegagalan kedua sering mengejutkan. Kita biasanya berpikir "makin banyak dokumen, makin aman". Kenyataannya, dokumen yang kontradiktif *lebih buruk* daripada tanpa dokumen sama sekali, karena agen akan memperlakukan informasi basi sebagai kebenaran.

1.2 Jawaban VCBD dalam dua kalimat

VCBD (*Vibe Coding Blueprint Drafter*) menjawab kegagalan pertama dengan prinsip **"gali dan konfirmasi dulu, baru tulis"**, dan kegagalan kedua dengan prinsip **"satu fakta, satu rumah"**. Setelah itu, blueprint yang lahir dari kedua prinsip tadi disambungkan ke tahap pembangunan dan audit dengan disiplin yang sama, sehingga klaim "sudah selesai" selalu didukung oleh perintah yang benar-benar dijalankan.

> **Analogi rumah.** Bayangkan Anda menyuruh tukang membangun rumah. **vcbd** adalah pembuat gambar kerja yang lengkap dan tidak saling bertentangan. **coding-vcbd** adalah mandor yang memastikan rumah dibangun sesuai gambar, satu ruangan demi satu ruangan. **review-vcbd** adalah pengawas yang memeriksa dengan bukti. Untuk rumah yang sudah berdiri tanpa gambar, **reverse-vcbd** dan **reverse-web-vcbd** mengukur ulang bangunannya lalu menggambar ulang gambar kerjanya.

1.3 Lima skill, satu siklus hidup

Suite VCBD versi 2.6 terdiri atas lima skill Claude Code. Masing-masing punya satu tanggung jawab, dan keluaran satu skill menjadi masukan skill berikutnya.

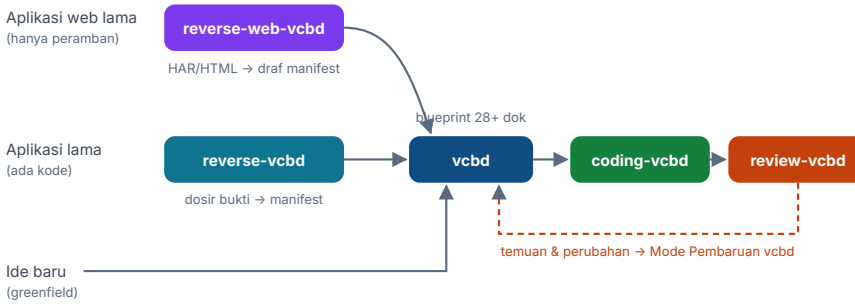


Diagram 1.1 — Alur kerja VCBD Suite dari tiga titik awal yang berbeda.

Skill	Peran	Mulai dari sini bila…
`vcbd`	Menyusun paket blueprint 28+ dokumen lewat wawancara bertahap	Aplikasi belum ada, baru ide
`coding-vcbd`	Mengeksekusi blueprint menjadi aplikasi, satu *vertical slice* demi slice	Blueprint sudah ada, coding belum mulai
`review-vcbd`	Mengaudit, menguji, melacak bug, dan menambal terhadap blueprint	Kode sudah ada, ingin tahu di mana ia menyimpang

Skill	Peran	Mulai dari sini bila…
`reverse-vcbd`	Membedah aplikasi yang ada kodenya menjadi dosir bukti, lalu blueprint	Aplikasi sudah ada, dokumennya tidak ada
`reverse-web-vcbd`	Membedah aplikasi web kotak hitam (HAR/HTML) menjadi dosir dan draf manifest	Aplikasinya hanya bisa dibuka lewat peramban

1.4 Empat konvensi bersama

Kelima skill bisa saling menyambung karena mereka mematuhi empat konvensi yang sama. Anda akan melihat keempatnya muncul berulang kali di bab-bab berikutnya.

1. **Satu fakta, satu rumah.** `07` adalah rumah skema, `11` rumah perintah, `06` rumah alur proses, `26` rumah fakta antarmuka. Dokumen lain cukup merujuk, tidak menyalin.
2. **Gerbang keras sebelum menulis.** `vcbd` menunggu konfirmasi Ringkasan Kebutuhan; `reverse-vcbd` dan `reverse-web-vcbd` menunggu konfirmasi Ringkasan Temuan.
3. **Gerbang mesin, bukan harapan.** `validate.sh`, `gerbang.sh`, `pindai.py`, dan uji regresi *menguji* hal yang biasanya hanya *diharapkan*.
4. **Tidak ada klaim tanpa bukti.** Fakta membawa `path:baris`, `[KODE:]`, `[DATA:]`, atau `[AMATI:]`. Yang belum pasti diberi label `[USULAN]`, `[ASUMSI]`, `[TERBUKA]`, atau `[ISI:]`.

1.5 Kapan VCBD cocok, dan kapan tidak

Kejujuran dulu: VCBD bukan untuk semua proyek. Paket 28 dokumen adalah investasi, dan investasi hanya masuk akal bila ada yang dilindungi.

Cocok untuk

- aplikasi yang akan dipelihara bertahun-tahun;
- aplikasi dengan banyak peran atau tabel;
- sistem lama yang mau dilanjutkan, tempat salah langkah itu mahal;
- tim yang bergantian memakai agen AI pada repo yang sama.

Kurang cocok untuk

- prototipe sekali buang;
- skrip kecil satu berkas;
- eksperimen yang belum jelas arah produknya.

Untuk kasus di kolom kanan, prompt langsung ke Claude Code biasanya lebih efisien. Bila prototipe itu ternyata bertahan hidup, Anda selalu bisa menyusulkan blueprint-nya dengan `reverse-vcbd` (Bab 6).

1.6 Cara membaca buku ini

Buku ini bergaya tutorial. Setiap bab skill memakai dua proyek demo yang benar-benar dijalankan saat buku ini disusun:

- **PRESQU**: aplikasi pencatatan kehadiran pegawai lewat kode QR harian, dibangun dari nol (Bab 3–5);
- **SI-CUTI**: aplikasi pengajuan cuti bergaya Laravel yang sudah "berjalan" tanpa dokumen, dibedah ulang (Bab 6–7).

Semua tangkapan layar terminal di buku ini adalah keluaran asli skrip VCBD v2.6, hanya dipangkas (*crop*) agar fokus. Lingkaran kuning bernomor 1 2 3 pada gambar menunjuk baris yang dibahas di teks sesudahnya.

Konvensi penulisan. Perintah yang diketik di terminal diawali `$`. Kalimat yang diketik ke Claude Code ditampilkan dalam kotak **PROMPT**. Nama berkas dan perintah memakai `huruf rata`. Angka dua digit seperti `07` atau `23` merujuk ke dokumen blueprint bernomor tersebut.

BAB 2

Menyiapkan Bengkel Kerja

Sebelum menyusun blueprint pertama, kita pasang kelima skill, memastikan semuanya terbaca oleh Claude Code, lalu memeriksa bahwa skrip gerbangnya bisa berjalan di mesin Anda.

Setelah membaca bab ini, Anda dapat:

- memasang VCBD Suite di Claude Code (lokal) maupun claude.ai;
- memahami mengapa kelima folder skill harus bersebelahan;
- memverifikasi instalasi dengan uji asap singkat.

2.1 Kebutuhan sistem

VCBD sengaja dibuat ringan. Seluruh skripnya hanya memakai pustaka standar Python dan Bash, jadi tidak ada `pip install` yang wajib.

Kebutuhan	Untuk	Catatan
Python 3.8+	Semua skrip `.py`	Hanya pustaka standar; diuji pada 3.12 dan 3.14
Bash	`validate.sh`, `validate-kontrak.sh`, `gerbang.sh`	Kompatibel bash 3.2 bawaan macOS
`zip`	`scripts/kemas.sh`	Hanya untuk membuat paket claude.ai
`pymysql` *(opsional)*	`reverse-vcbd` membaca MySQL/MariaDB	Cadangan: klien `mysql` di PATH

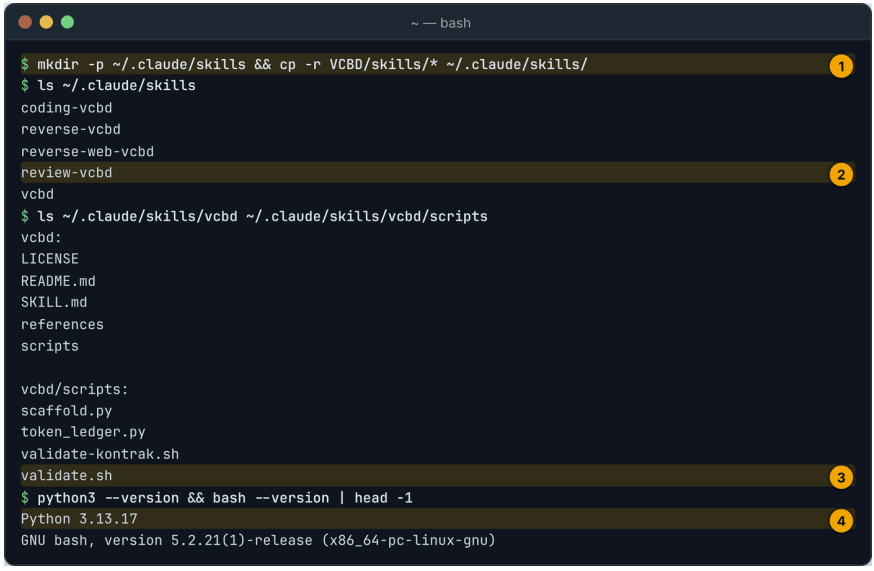
Kebutuhan	Untuk	Catatan
`psycopg2-binary` *(opsional)*	`reverse-vcbd` membaca PostgreSQL	SQLite tidak butuh tambahan
Claude Code	Menjalankan skill	CLI, aplikasi desktop, web, atau ekstensi IDE

2.2 Memasang di Claude Code

Langkah 1 Clone repositori, lalu salin isi folder `skills/` ke folder skill global Anda, `~/.claude/skills/`.

```
$ git clone https://github.com/Syamsuddin/VCBD.git
$ mkdir -p ~/.claude/skills && cp -r VCBD/skills/*
~/.claude/skills/
```

Gambar berikut memperlihatkan hasilnya pada mesin uji.



Gambar 2.1 — Memasang kelima skill ke `~/.claude/skills` dan memeriksa prasyaratnya.

1. Perintah salin. Untuk instalasi per proyek, ganti tujuannya menjadi `.claude/skills/` di root proyek.

2. Kelima folder harus muncul berdampingan dengan nama persis seperti ini.
3. Setiap skill membawa `SKILL.md` (instruksi untuk agen), `references/` (dibaca saat dibutuhkan), dan `scripts/` (dieksekusi, bukan dibaca).
4. Versi Python dan Bash cukup diperiksa sekali.

Langkah 2 Mulai sesi Claude Code baru. Skill baru terbaca otomatis. Bila belum muncul, keluar lalu buka lagi sesinya.

2.3 Mengapa harus bersebelahan?

Beberapa skrip dipakai lintas skill. Daripada menyalin kode yang sama ke lima tempat (yang akan melanggar prinsip "satu fakta, satu rumah"), skill di hilir mencari skrip di folder saudaranya.

Skill	Memakai dari skill lain
`reverse-vcbd`	`vcbd/scripts/scaffold.py` dan `validate.sh`
`coding-vcbd`	`vcbd/scripts/token_ledger.py` dan `validate.sh`
`review-vcbd`	`coding-vcbd/scripts/gerbang.sh` dan `meter.py`
`reverse-web-vcbd`	`vcbd` (lewat draf manifest)

> **Jebakan umum.** Memasang sebagian skill saja, misalnya hanya `coding-vcbd`, membuat gerbang mesinnya gagal mencari `validate.sh`. Pasang kelimanya, walaupun Anda baru berencana memakai satu.

2.4 Menyalin skrip ke proyek

`scaffold.py` otomatis menyalin `validate.sh` dan `token_ledger.py` ke folder `scripts/` proyek saat blueprint dibuat. Salin juga skrip `coding-vcbd` dan `review-vcbd`, supaya proyek bisa memeriksa dirinya sendiri tanpa bergantung pada folder skill:

```
$ cd /path/proyek
$ cp ~/.claude/skills/coding-vcbd/scripts/{meter.py,gerbang.sh}
scripts/
$ cp ~/.claude/skills/review-vcbd/scripts/*.py scripts/
```

> **Selalu jalankan gerbang dari root proyek.** `validate.sh`, `validate-kontrak.sh`, dan `gerbang.sh` membaca berkas secara relatif. Bila dijalankan dari folder lain, ketiganya melapor `[FAIL] … jalankan skrip dari root proyek`.

2.5 Memasang di claude.ai

Di claude.ai, skill diunggah sebagai berkas zip. Zip per skill tersedia di halaman rilis v2.6, atau bisa Anda buat sendiri dari root repositori:



Gambar 2.2 — Mengemas lima zip skill untuk diunggah ke claude.ai.

1. `kemas.sh` menyusun tiap folder skill menjadi zip yang berisi `SKILL.md`, `README.md`, `LICENSE`, `references/`, dan `scripts/`.
2. Kelima zip ini diunggah satu per satu lewat **Settings → Capabilities → Skills**.

Unggah **kelimanya**. Di claude.ai, `reverse-vcbd` mencari `vcbd` di `/mnt/skills/user/vcbd/`. Bila akun Anda sudah punya skill bernama sama, hapus dulu versi lamanya supaya tidak ada dua versi yang berbeda.

2.6 Bila salinan skill ganda

Skill yang diunggah ke akun claude.ai ikut tersinkron ke Claude Code dan tampil sebagai `anthropic-skills:<nama>`, di samping salinan lokal. Bila versinya berbeda, agen bisa memakai versi yang salah. Solusi yang disarankan adalah menyamakan keduanya. Alternatifnya, blokir salinan sinkron di `~/.claude/settings.json`:

```
{
  "permissions": {
    "deny": [
      "Skill(anthropic-skills:vcbd)",
      "Skill(anthropic-skills:coding-vcbd)",
      "Skill(anthropic-skills:review-vcbd)",
      "Skill(anthropic-skills:reverse-vcbd)",
      "Skill(anthropic-skills:reverse-web-vcbd)"
    ]
  }
}
```

2.7 Memeriksa instalasi

Tiga perintah berikut memastikan semua skrip valid secara sintaks dan uji regresi `reverse-web-vcbd` lulus:

```
$ cd ~/.claude/skills
$ for f in vcbd/scripts/*.sh coding-vcbd/scripts/*.sh; do bash -n
"$f" && echo "OK $f"; done
$ for f in */scripts/*.py; do python3 -m py_compile "$f" && echo
"OK $f"; done
$ python3 reverse-web-vcbd/uji/uji_regresi.py      # harus: 32/32
lulus
```

Anda akan melihat hasil uji regresi itu di Bab 7. Untuk sekarang, cukup pastikan tidak ada baris yang gagal.

2.8 Memperbarui dan mencopot

```
$ cd VCBD && git pull
$ for d in vcbd coding-vcbd review-vcbd reverse-vcbd reverse-web-
vcbd; do
    rm -rf ~/.claude/skills/$d && cp -r skills/$d
~/.claude/skills/
  done
```

Proyek yang sudah menyalin skrip ke `scripts/` tidak ikut terbarui otomatis. Salin ulang bila `CHANGELOG.md` menyebut ada perubahan skrip. Untuk mencopot, cukup hapus kelima folder skill. Berkas yang sudah lahir di proyek (`docs/`, `CLAUDE.md`, `INDEX.md`, `scripts/`) tidak tersentuh.

Rangkuman Bab 2

- VCBD hanya butuh Python 3.8+ dan Bash; dependensi lain opsional.
- Pasang kelima skill sekaligus dan biarkan foldernya bersebelahan.
- Salin skrip gerbang ke `scripts/` proyek agar proyek bisa memeriksa dirinya sendiri.
- Jalankan setiap skrip gerbang dari root proyek.

BAB 3

vcbd: Menyusun Blueprint

Skill `vcbd` adalah akar rantai. Ia mengubah ide aplikasi menjadi paket blueprint 28+ dokumen yang konsisten, murah dimuat, dan bisa diverifikasi mesin. Di bab ini kita menyusun blueprint aplikasi PRESQU dari nol.

Setelah membaca bab ini, Anda dapat:

- menjalankan lima fase `vcbd`, dari triase sampai serah terima;
- membaca struktur paket blueprint dan peran tiap klaster dokumen;
- memakai `scaffold.py` dan `validate.sh` sebagai gerbang mesin;
- memakai Mode Pembaruan, mode SPLIT, dan profil native.

3.1 Empat Hukum vcbd

Seluruh perilaku skill ini diturunkan dari empat hukum yang tidak bisa ditawar. Hafalkan keempatnya; setiap kali skill "menolak" permintaan Anda, alasannya hampir selalu salah satu dari ini.

1. **Tidak ada generasi tanpa konfirmasi.** Skill mewawancarai Anda, menyajikan Ringkasan Kebutuhan, lalu menunggu jawaban "ya" sebelum menulis satu dokumen pun. Permintaan "langsung saja, tidak usah tanya" tidak membatalkan hukum ini; ia hanya memperpendek wawancara menjadi satu ringkasan asumsi dan satu kali konfirmasi.

2. **Satu fakta, satu rumah.** Setiap fakta ditulis penuh di satu dokumen pemiliknya. Dokumen lain cukup merujuk. Skema hanya hidup di `07`, istilah domain di `04`, perintah di `11`.

3. **INDEX me-rute, CLAUDE.md inti.** `INDEX.md` adalah router (jenis task → dokumen yang dimuat). `CLAUDE.md` adalah inti yang selalu aktif. Dengan begitu 28 dokumen tidak perlu dimuat sekaligus.

4. **Setiap baris membayar dirinya dan dapat diverifikasi.** Sebelum menulis baris apa pun: *"jika dihapus, apakah agen akan keliru?"* Kriteria terima harus berupa perintah dengan sinyal lulus/gagal, bukan "pastikan berfungsi".

> **Scaffold tetap, isi adaptif.** "28 dokumen" adalah *kerangka alamat* yang tetap, bukan kewajiban menulis 28 halaman penuh. Dokumen yang isinya 100% mengikuti default didaftarkan di field `collapsed` manifest, dan stub-nya ditulis otomatis oleh skrip. Alamatnya stabil untuk agen, sementara isinya tetap hemat.

3.2 Anatomi paket blueprint

Paket VCBD terdiri atas dua berkas di root proyek dan dokumen bernomor di `docs/`.

Klaster	Dokumen	Fungsi singkat
Root	`CLAUDE.md`, `INDEX.md`	Inti selalu-aktif dan router konteks
Strategis	00–03	Ringkasan eksekutif, PRD, scope, roadmap (fase berupa *vertical slice*)
Domain	04–07	Glosarium, peran & matriks akses, proses bisnis, model data
Fondasi teknis	08–12	Arsitektur, stack, lingkungan dev, perintah, struktur proyek
Kualitas & operasi	13–16	Pengujian, penanganan galat, observabilitas, panduan debugging

Klaster	Dokumen	Fungsi singkat
Perilaku agen	17–22	Alur kerja agen, aturan perbaikan, templat task, guardrail, keamanan, kebijakan perubahan
Gerbang selesai	23–25	Kriteria terima per fitur, definisi selesai, checklist rilis
Kondisional	26, 27	`26_UI_CONVENTIONS` untuk proyek ber-UI; `27_API_CONTRACT` untuk mode SPLIT

Daftar lengkap ke-28 dokumen beserta pemilik faktanya ada di Lampiran A.

3.3 Fase 0: Triase

Semuanya dimulai dari satu prompt sederhana. Anda cukup memberi nama dan deskripsi umum.

PROMPT

Siapkan blueprint lengkap aplikasi PRESQU: pencatatan kehadiran pegawai via pindai kode QR harian. Satu kantor, dua peran (admin dan pegawai). Saya ingin Python murni tanpa framework, basis data SQLite.

Pada fase triase, skill **mengumpulkan fakta tanpa bertanya**. Bila repo sudah punya `composer.json` atau `pubspec.yaml`, versi framework dibaca dari sana, bukan ditanyakan. Lalu skill menetapkan lima hal:

1. **Greenfield atau brownfield.** PRESQU dibangun dari nol, jadi greenfield.
2. **Ber-UI atau tidak.** PRESQU punya halaman pindai, jadi `ui.enabled = true`. Kategori wawancara K9 dan dokumen `26` ikut aktif.

3. **Mode SPLIT atau tunggal.** Satu deployable kecil, jadi tunggal. Skill tidak memaksakan pemisahan backend/frontend pada aplikasi kecil.
4. **Profil framework.** "Python murni tanpa framework" adalah keputusan sah. Skill merekam `stack.framework = "none"` dan mengaktifkan **profil native**: dokumen 12, 14, dan 21 wajib ditulis penuh karena blueprint memikul peran yang biasanya ditanggung framework.
5. **Jebakan (landmines)** untuk brownfield. Tidak relevan di PRESQU.

> **Skill tidak akan "memperbaiki" pilihan Anda.** Bila Anda memilih tanpa framework, skill tidak diam-diam mengusulkan Flask atau Django. Pilihan sengaja dihormati, dan "framework" justru masuk daftar teknologi terlarang di `09`.

3.4 Fase 1: Wawancara penggalian

Wawancara memakai bank pertanyaan sepuluh kategori. Aturan mainnya: maksimal 3–4 pertanyaan per ronde, setiap pertanyaan disertai **usulan default**, dan hal yang sudah diketahui tidak ditanyakan ulang.

Kode	Kategori	Mengisi dokumen
K1	Identitas & tujuan	00, 01
K2	Fitur & scope	01, 02, 03
K3	Aktor & peran	05, 06, 21
K4	Domain & data	04, 07
K5	Stack & arsitektur	08, 09, 12
K6	Lingkungan & operasi	10, 11, 15
K7	Kualitas & keamanan	13, 14, 21, 23, 24
K8	Kebijakan perubahan & risiko	18, 20, 22, 25

Kode	Kategori	Mengisi dokumen
K9	Antarmuka (kondisional ber-UI)	26
K10	Kontrak backend ↔ frontend (kondisional split)	27 + `kontrak/`

Contoh satu ronde pertanyaan yang disertai default:

> **Claude:** K3 — Peran. Saya sarankan dua peran: *admin* (kelola pegawai, lihat semua rekap, ekspor) dan *pegawai* (pindai, lihat rekap sendiri). Setuju? · K4 — Data sensitif: NIP saya tandai PII dan data kehadiran tanpa *hard-delete*. Setuju?

> **Anda:** Setuju semua. Tambahkan: pegawai tidak boleh melihat rekap pegawai lain.

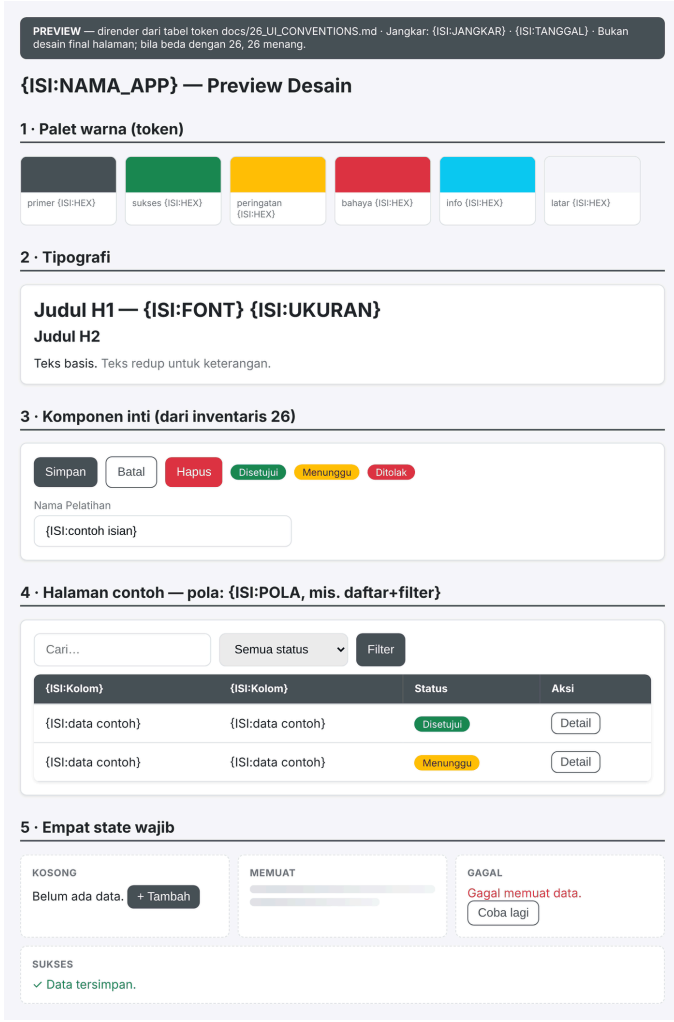
Banyak dokumen (16–20, 22–25) tidak perlu pertanyaan terpisah. Skill menurunkannya dari jawaban lain ditambah konvensi, lalu meminta Anda mengonfirmasi bloknya sekaligus.

K9: kata sifat tidak bisa dieksekusi

Untuk proyek ber-UI, K9 adalah kategori yang paling sering diremehkan. "Modern, bersih, profesional" bukan spesifikasi. `#0F4C81`, radius 8 px, Inter 16/1.5 *adalah* spesifikasi. Tugas K9 adalah mengompilasi selera menjadi token yang bisa diuji mesin:

1. **Jangkar desain** dulu: design system yang sudah ada, aplikasi acuan, atau identitas instansi.
2. **Token inti** dalam tabel: warna (hex), huruf dan skala, radius, jarak. Aturan yang dikunci: nilai di luar tabel **terlarang**.
3. **Inventaris halaman → pola** (daftar, detail, formulir, dasbor, autentikasi).
4. **Empat state wajib** untuk setiap halaman ber-data: kosong, memuat, gagal, sukses.
5. **Larangan UI**: gradien tanpa alasan, emoji di antarmuka, ikon dari set berbeda-beda, dan seterusnya.

Setelah token terkunci, skill menawarkan **sekali** preview desain. Preview dirender dari kerangka HTML beku; model hanya menyuntikkan nilai token.



Gambar 3.1 — Kerangka `contoh-preview.html` bawaan skill, dirender apa adanya. Hanya blok token `:root` dan teks `{ISI:}` yang diganti saat preview dibuat.

3.5 Fase 2: Konfirmasi (gerbang keras)

Wawancara ditutup dengan **Ringkasan Kebutuhan**: tabel padat per kategori, ditambah dua daftar terpisah.

```
## Ringkasan Kebutuhan — PRESQU
| Kategori         | Ringkas
|
|------------------|-----------------------------------------------
--------|
| Tujuan & metrik  | Hentikan absensi manual; 100% digital, < 5
dtk/scan  |
| Fitur MVP        | Scan QR masuk/pulang; rekap harian; ekspor
CSV       |
| Out-of-scope     | Payroll, cuti online, mobile native, multi-
kantor    |
| Peran & akses    | admin (kelola, semua rekap, ekspor); pegawai
(scan)  |
| Stack            | Python 3.12 stdlib + SQLite; Terlarang:
framework    |
| …                | …
|

[ASUMSI]  Jendela pindai masuk 06:00–09:00, pulang 14:00–18:00
WITA
[TERBUKA] Notifikasi keterlambatan via WhatsApp — pasca-MVP

Konfirmasi: generate 28 dokumen blueprint dengan ringkasan di
atas?
Jawab "ya" atau koreksi bagian yang salah.
```

`[ASUMSI]` dipakai bila tidak Anda koreksi. `[TERBUKA]` sengaja ditunda. Koreksi memperbarui ringkasan, lalu skill meminta konfirmasi ulang hanya untuk bagian yang berubah. **Tidak ada dokumen yang ditulis sebelum Anda menjawab "ya".**

3.6 Fase 3: Generasi

Langkah 1: manifest lebih dulu

Sebelum dokumen apa pun, skill menulis `docs/_MANIFEST.json`: state bersama yang merekam ringkasan terkonfirmasi, peta pemilik fakta, dan graf rujukan.

docs/_MANIFEST.json

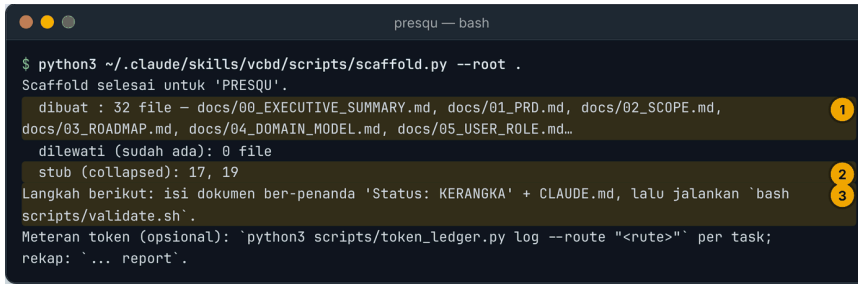
```
{
  "app": {"name": "PRESQU", "description": "Pencatatan kehadiran pegawai via QR", "type": "greenfield"},
  "ui": {"enabled": true, "anchor": "Dasbor instansi: tenang, kontras tinggi"},   (1)
  "split": {"enabled": false},
  "canonical_owners": {"schema": "07", "roles": "05", "commands": "11", "repair_rules": "18", "guardrails": "20", "security": "21"},
  "referenced_by": {"07": ["06", "21", "23"], "05": ["06", "23"], "11": ["13", "23", "25", "CLAUDE"], "26": ["23", "CLAUDE"]},   (2)
  "collapsed": ["17", "19"],   (3)
  "landmines": [],
  "assumptions": ["Jendela scan masuk 06:00-09:00, pulang 14:00-18:00 WITA"],
  "requirements": { … }
}
```

Gambar 3.2 — Potongan `_MANIFEST.json` PRESQU (dipadatkan).

1. `ui.enabled` memicu `scaffold.py` menulis dokumen `26` dan rute UI di `INDEX.md`.
2. `referenced_by` adalah graf pemilik → perujuk. Bila skema (`07`) berubah, dokumen 06, 21, dan 23 ikut ditinjau. Graf inilah yang dipakai Mode Pembaruan.
3. `collapsed` mendaftar dokumen yang sengaja menjadi stub (di sini 17 dan 19).

Langkah 2: kerangka deterministik dengan scaffold.py

Skill tidak menulis `INDEX.md`, stub, atau kerangka dokumen dengan tangan. Itu membuang token dan rawan perbedaan format. Semuanya ditulis oleh skrip:

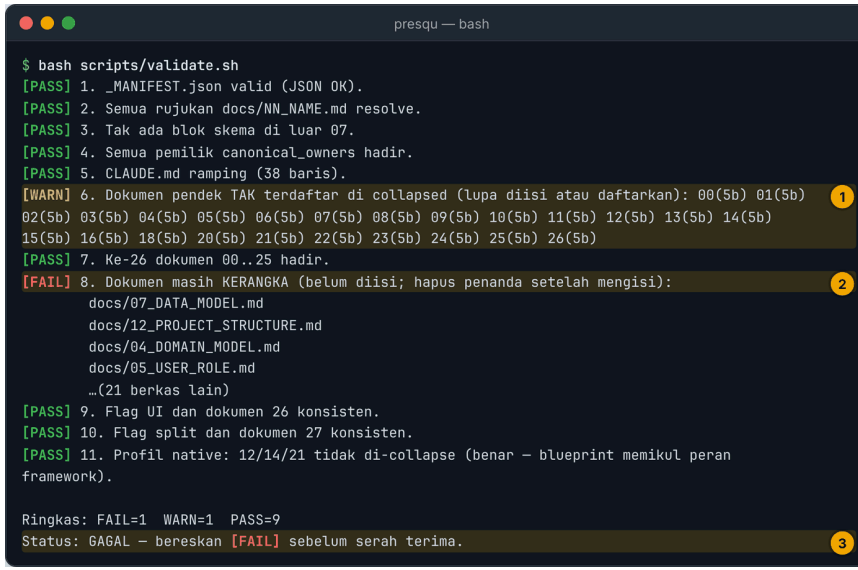


Gambar 3.3 — Menjalankan `scaffold.py` pada proyek PRESQU.

1. 32 berkas terbentuk: dokumen 00–26, `CLAUDE.md`, `INDEX.md`, ledger token, serta salinan `validate.sh` dan `token_ledger.py` di `scripts/`.
2. Dokumen 17 dan 19 langsung ditulis sebagai stub final karena terdaftar di `collapsed`.
3. Skrip memberi tahu langkah berikutnya. Skrip bersifat anti-timpa: berkas yang sudah ada dilewati, kecuali dengan `--force`.

Langkah 3: gerbang mesin menangkap kerangka kosong

Coba jalankan validator sekarang, sebelum dokumen diisi. Inilah gunanya gerbang mesin: ia tidak bisa dibujuk.



Gambar 3.4 — `validate.sh` menolak paket yang masih berupa kerangka.

1. `[WARN] 6`: dokumen yang terlalu pendek dan tidak terdaftar di `collapsed`. Validator bertanya: lupa diisi, atau memang stub?
2. `[FAIL] 8`: masih ada penanda `Status: KERANGKA`. Ini yang mencegah paket setengah jadi diserahterimakan.
3. Status akhir `GAGAL` dan *exit code* 1. Agen tidak boleh lanjut ke serah terima.

Langkah 4: mengisi substansi

Skill mengisi tiap dokumen dari manifest dengan rubrik ekonomi token: tabel lebih baik daripada narasi, path ditulis eksplisit, dan blok verifikasi konkret. Contoh dokumen pemilik skema:

docs/07_DATA_MODEL.md

```
# 07 — DATA_MODEL

## Tabel `employees`                                                    (1)
| Kolom | Tipe | Constraint |
|---|---|---|
| id | INTEGER | PK |
| nip | TEXT | UNIQUE, NOT NULL (PII) |
| name | TEXT | NOT NULL |
| role | TEXT | CHECK IN ('admin','pegawai') |
## Tabel `attendances`
| Kolom | Tipe | Constraint |
|---|---|---|
| id | INTEGER | PK |
| employee_id | INTEGER | FK employees.id |
| type | TEXT | CHECK IN ('masuk','pulang') |
| scanned_at | TEXT | ISO-8601, NOT NULL |
UNIQUE (employee_id, type, date(scanned_at)). Tanpa hard-delete.          (2)
```

Gambar 3.5 — `07_DATA_MODEL.md`: satu-satunya rumah skema di seluruh paket.

1. Kolom, tipe, dan constraint ditulis dalam tabel. Dokumen lain yang butuh skema cukup menulis "lihat `docs/07_DATA_MODEL.md`".
2. Aturan bisnis yang punya padanan di basis data (keunikan satu "masuk" per hari) ditulis sebagai constraint, bukan prosa.

Dokumen kriteria terima menentukan kapan sebuah fitur boleh disebut selesai. Perhatikan blok verifikasinya: perintah dan sinyal lulusnya ditulis eksplisit.

docs/23_ACCEPTANCE_CRITERIA.md

```
# 23 — ACCEPTANCE_CRITERIA

## F1 — Scan QR masuk/pulang                                   (1)
- [ ] Kode sah dalam jendela → 201, tercatat di attendances.
- [ ] Kode salah → 422 "Kode tidak berlaku".
- [ ] Di luar jendela → 422 "Di luar jam pindai".
- [ ] Pindai ganda jenis sama hari ini → 409.
- [ ] Halaman /scan menampilkan empat state.
### Verifikasi                                                (2)
`python3 -m unittest tests.test_scan` → OK, 0 failures.
```

Gambar 3.6 — Blok fitur F1 di `23_ACCEPTANCE_CRITERIA.md`.

1. Satu blok per fitur MVP. Setiap butir bisa dibuktikan benar atau salah.
2. Blok verifikasi berisi perintah yang akan dijalankan `coding-vcbd` beserta sinyal lulusnya.

Untuk proyek ber-UI, dokumen `26` mengunci token desain:

docs/26_UI_CONVENTIONS.md

```
# 26 — UI_CONVENTIONS

Jangkar desain: dasbor instansi — tenang, kontras tinggi.
## Token (BEKU — nilai di luar tabel terlarang)              (1)
| Token | Nilai |
|---|---|
| --warna-primer | #0F4C81 |
| --warna-aksen | #F2A900 |
| --warna-latar | #F7F8FA |
| --warna-galat | #C62828 |
| --huruf | Inter, 16px/1.5 |
| --radius | 8px |
| --jarak | 4px · 8px · 16px · 24px |
## Empat state wajib                                          (2)
kosong · memuat · gagal · sukses
## Larangan                                                   (3)
Tanpa gradien; tanpa warna di luar token; tanpa ikon tanpa label teks.
```

Gambar 3.7 — `26_UI_CONVENTIONS.md`: token beku, empat state, larangan.

1. Tabel token. Warna atau ukuran di luar tabel ini terlarang dan akan menjadi temuan saat audit (Bab 5).
2. Empat state wajib untuk setiap halaman ber-data.
3. Larangan UI yang dikonfirmasi saat wawancara K9.

Langkah 5: CLAUDE.md dan INDEX.md

`CLAUDE.md` sengaja ramping; validator memberi peringatan bila lebih dari 50 baris.

CLAUDE.md

```
# CLAUDE.md

File ini selalu aktif. Untuk hal di luar ini → buka INDEX.md.

## Proyek
PRESQU — absensi pegawai via pindai QR harian. Detail: docs/00_EXECUTIVE_SUMMARY.md. Scope: docs/02_SCOPE.md.

## Prinsip kerja agen (non-negotiable)
1. Friksi sebanding irreversibilitas.
2. Lapisan deterministik (validasi, constraint, tes) di bawah penalaran.
3. Human-in-the-loop untuk high-stakes (migrasi DB, hapus data, keamanan, rilis).
4. Jangan refactor keputusan yang disengaja diam-diam.
5. Hormati scope (docs/02). Out-of-scope → berhenti & tanya.

## Stack  (sumber: docs/09)
Python 3.12 stdlib + SQLite, tanpa framework — Terlarang: jQuery, framework web

## Struktur & konvensi  (sumber: docs/12)
SQL hanya di app/repo.py · handler tipis → service → repo · snake_case

## Perintah penting  (sumber: docs/11)
test: `python3 -m unittest discover -s tests` · run: `python3 -m app.server` · migrasi ⚠️: `python3 -m app.migrate`

## Guardrail inti  (penuh: docs/20, 21, 22)
Jangan tambah framework · jangan hapus data kehadiran · jangan tulis NIP ke log · jangan skip tes

## Antarmuka  (sumber: docs/26_UI_CONVENTIONS.md)
Dasbor instansi tenang; tanpa gradien, tanpa warna di luar token 26, empat state wajib

## Alur per-task  (penuh: docs/17, 19)
Baca INDEX.md → muat dokumen relevan → konfirmasi scope → vertical slice → tes (docs/13) → DoD (docs/24).

## Definisi selesai (ringkas; penuh: docs/24)
23 terpenuhi + suite hijau + `bash scripts/gerbang.sh` exit 0

## Untuk apa pun di luar ini → INDEX.md
```

Gambar 3.8 — `CLAUDE.md` PRESQU setelah slot `[ISI:]` diisi.

1. Lima prinsip kerja agen yang tidak bisa ditawar, termasuk "friksi sebanding irreversibilitas".
2. Hanya perintah tersering, dengan simbol peringatan pada migrasi. Daftar lengkap tetap di `11`.
3. Untuk apa pun di luar inti, agen diarahkan ke `INDEX.md`.

`INDEX.md` ditulis utuh dan final oleh `scaffold.py`. Model tidak menyuntingnya.

INDEX.md

```
# INDEX — Peta Konteks
Untuk AI: baca ini sebelum task. Muat hanya dokumen yang ditunjuk. JANGAN muat semua.

## Tier 0 — selalu aktif
CLAUDE.md (inti). Jangan baca ulang sumbernya kecuali butuh detail.
Esensi 17_AGENT_WORKFLOW (alur per-task) & 19_TASK_TEMPLATE (format task) sudah diringkas
di CLAUDE.md — keduanya sengaja TIDAK ada di rute task. Muat dokumen penuhnya hanya saat
butuh detail alur/format, bukan tiap task.

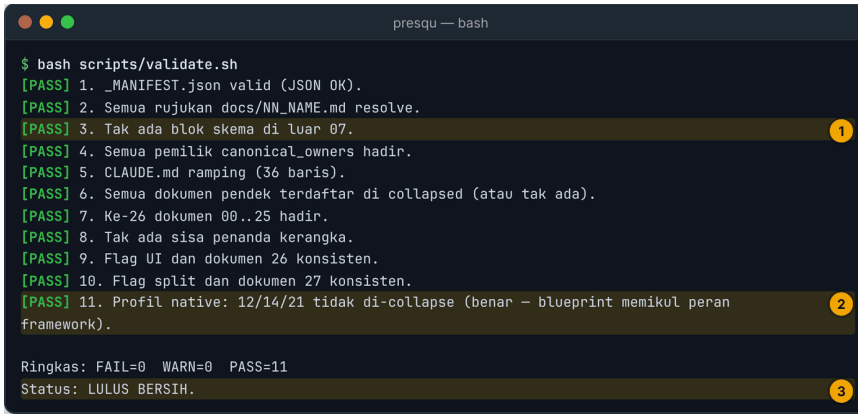
## Rute: jenis task → dokumen                                                          (1)
| Jenis task | Muat (inti) | Kondisional / catatan |
|---|---|---|
| Fitur baru (vertical slice) | 01,02,06,07,11,23 | cek 02 dulu; +04 istilah domain ambigu;   (2)
+05 sentuh peran/izin; +13 jenis tes baru; format task: 19 (esensi di CLAUDE); +26 bila
menyentuh tampilan |
| Halaman/komponen UI baru | 02,11,23,26 | +06 bila alur baru; +05 bila sentuh peran/izin;
empat state wajib (26) |
| Ubah skema DB / migrasi | 07,11,13,22 | ⚠️ irreversibel; 22 wajib; +04 bila          (3)
entitas/istilah baru |
| Perbaikan bug | 11,13,14,16,18 | jangan lewat scope bug; 11 utk repro+tes regresi |
| Endpoint/API baru | 06,07,08,11,13,21 | 21 bila sensitif |
| Perubahan keamanan | 15,20,21,22 | ⚠️ gerbang manusia |
| Tambah peran/izin | 05,06,21 | — |
| Refactor arsitektur | 04,08,22 | jangan ubah keputusan sengaja |
| Observability/logging | 14,15,16 | — |
| Setup lingkungan | 09,10,11,12 | — |
| Rilis | 11,15,22,25 | ⚠️ ikuti 25 berurutan; +24 bila butuh DoD rinci |
| Strategi/scope | 00,01,02,03 | tanpa kode |
```

Gambar 3.9 — Rute konteks di `INDEX.md`: jenis task menentukan dokumen yang dimuat.

1. Tabel rute adalah jantung hemat token: agen hanya memuat dokumen di kolom "Muat".
2. Untuk fitur baru, cukup enam dokumen: 01, 02, 06, 07, 11, 23. Sisanya kondisional.
3. Task irreversibel seperti migrasi skema diberi simbol peringatan dan wajib memuat `22`.

Langkah 6: pemangkasan dan validasi akhir

Lintasan terakhir menghapus baris yang gagal uji Hukum 4 dan memastikan tidak ada fakta yang tertulis dua kali. Setelah itu validator dijalankan lagi:



Gambar 3.10 — Paket PRESQU lulus sebelas cek konsistensi.

1. Cek 3 memastikan tidak ada blok skema di luar `07`. Inilah Hukum 2 yang diuji mesin.
2. Cek 11 khusus profil native: dokumen 12, 14, dan 21 tidak boleh di-*collapse*.
3. `LULUS BERSIH`: tidak ada FAIL maupun WARN.

Sebelas cek validate.sh. (1) manifest JSON valid · (2) semua rujukan `docs/NN_NAME.md` resolve · (3) tidak ada blok skema di luar 07 · (4) pemilik kanonik hadir · (5) `CLAUDE.md` ramping · (6) dokumen pendek terdaftar di `collapsed` · (7) dokumen 00–25 hadir · (8) tidak ada sisa KERANGKA · (9) flag UI ↔ 26 · (10) flag split ↔ 27 · (11) profil native.

3.7 Fase 4: Serah terima

Setelah validator lulus, skill menyampaikan cara memakai blueprint. Kalimat pembuka sesi baru selalu sama:

PROMPT

Baca CLAUDE.md lalu INDEX.md. Untuk task [nama task], muat hanya dokumen yang ditunjuk INDEX. Kerjakan, lalu jalankan blok verifikasinya.

Tiga pengingat lain disampaikan pada tahap ini: (1) saat satu fakta berubah, perbarui dokumen pemiliknya dan manifest, lalu jalankan `validate.sh`; (2) setelah implementasi, minta subagen dengan konteks segar membandingkan diff terhadap dokumen; (3) meteran token opsional tersedia lewat `token_ledger.py`.

3.8 Mode Pembaruan

Bila paket sudah ada dan Anda minta perubahan, skill tidak mewawancarai ulang dari nol. Ia hanya menanyakan delta-nya, memperbarui manifest, lalu meregenerasi dokumen yang terdampak menurut graf `referenced_by`, bukan menurut tebakan dari prosa.

PROMPT

Ubah blueprint PRESQU: tambahkan kolom `unit_kerja` pada pegawai, dan rekap harian bisa difilter per unit kerja.

Pada contoh ini, pemilik faktanya adalah `07`. Graf `referenced_by` menyebut 06, 21, dan 23 sebagai perujuk, jadi keempatnya ditinjau. Konfirmasi tetap berlaku untuk delta, dan `validate.sh` wajib lulus lagi.

> **Paket warisan v1.2.** Proyek lama berformat 29 dokumen (`26_UI_DESIGN.md`) dimigrasikan lewat Mode Pembaruan: berkas diganti nama menjadi `26_UI_CONVENTIONS.md`, isinya diterjemahkan ke tabel token, lalu bagian yang belum ada dilengkapi.

3.9 Mode SPLIT: backend dan frontend terpisah

Bila frontend adalah deployable terpisah (aplikasi Flutter atau SPA) atau dikerjakan di sesi berbeda, aktifkan mode SPLIT. Dua agen di dua sesi tidak berbagi ingatan. Satu-satunya jembatan

mereka adalah kontrak.

```
monorepo/
├── kontrak/
│   ├── openapi.yaml        ← rumah tunggal endpoint + payload
(SemVer)
│   ├── KONTRAK.md          ← presedensi, glosarium bersama,
envelope error
│   └── scripts/validate-kontrak.sh
├── backend/   (CLAUDE.md, INDEX.md, docs/ — split.role = backend)
└── frontend/  (CLAUDE.md, INDEX.md, docs/ — split.role =
frontend, 07 collapsed)
```

Urutannya: `scaffold.py --init-kontrak` sekali dari root monorepo, dua manifest dengan pin `split.contract_version` yang sama, lalu scaffold dan validasi per paket. Gerbang lintas paket `validate-kontrak.sh` memeriksa lima hal: kontrak sehat, kedua paket ber-`split.enabled` dengan peran yang saling melengkapi, pin versi kedua manifest sama, pin itu sama dengan `info.version` di `openapi.yaml`, dan `27` hadir di kedua paket.

> **Jangan memecah aplikasi kecil "biar rapi".** Monolit ber-template di server tidak butuh dua paket. Seremoni ganda tanpa nilai adalah anti-pattern yang dicatat sendiri oleh skill.

3.10 Anti-pattern yang dicegah aktif

- Menulis dokumen sebelum konfirmasi.
- Menyalin fakta yang sama ke beberapa dokumen.
- Paket tanpa INDEX, atau `CLAUDE.md` yang gemuk.
- Verifikasi kabur seperti "uji menyeluruh".
- Mengarang fakta tanpa label `[ASUMSI]`.
- Roadmap horizontal per lapisan; yang benar adalah *vertical slice* yang bisa didemokan.
- Nilai UI di luar tabel token, atau kata sifat estetika tanpa token konkret.

- Brownfield: menulis 07/09/11/12 dari jawaban wawancara alih-alih mengekstraknya dari kode.

Latihan 3

1. Susun blueprint untuk aplikasi kecil milik Anda sendiri. Catat berapa ronde wawancara yang Anda butuhkan.
2. Jalankan `validate.sh` sebelum dan sesudah pengisian. Bandingkan hasilnya dengan gambar di bab ini.
3. Sengaja salin tabel skema ke `06`, lalu jalankan validator. Cek mana yang menangkapnya?

Rangkuman Bab 3

- `vcbd` berjalan dalam lima fase: triase, wawancara, konfirmasi, generasi, serah terima.
- Manifest ditulis lebih dulu; kerangka ditulis skrip; isi ditulis model; konsistensi diuji validator.
- `INDEX.md` membuat paket besar tetap murah dimuat.
- Mode Pembaruan memakai graf `referenced_by` untuk tahu dokumen mana yang terdampak.

BAB 4

coding-vcbd: Membangun Slice demi Slice

Blueprint yang bagus belum menjadi aplikasi. Skill `coding-vcbd` *mengeksekusinya slice demi slice, dengan pemuatan dokumen yang hemat dan gerbang selesai yang diuji mesin. Di bab ini kita membangun fitur pertama PRESQU: pindai QR masuk/pulang.*

Setelah membaca bab ini, Anda dapat:

- menjalankan tujuh fase `coding-vcbd`, dari kunci sesi sampai aplikasi selesai;
- membaca dan menindaklanjuti delapan syarat `gerbang.sh`;
- membaca counter waktu dan token dengan jujur;
- menerbitkan kontrak serah-terima untuk auditor.

4.1 Dua kegagalan yang dicegah

Agen yang mengarang. Blueprint sudah menjawab skema, perintah, konvensi, dan kriteria terima. Agen yang menebak ulang menghasilkan kode yang benar secara sintaks tetapi salah secara kebutuhan.

Agen yang mengaku selesai. "Sudah saya implementasikan" tanpa keluaran perintah adalah klaim, bukan bukti. Di `coding-vcbd`, selesai berarti `gerbang.sh` berakhir dengan *exit* 0.

4.2 Empat Hukum Eksekusi

1. **Blueprint adalah sumber kebutuhan; kode aktual adalah sumber keadaan.** Bila dokumen berbeda dengan kode, **kode menang**. Agen berhenti, melaporkan selisihnya, dan meminta keputusan. Ia tidak diam-diam memilih salah satu.
2. **Tidak ada kode di luar rencana slice.** Satu waktu satu slice, dengan daftar berkas yang diumumkan di muka. Berkas di luar daftar berarti berhenti dan minta izin.
3. **Selesai = diuji, bukan diakui.** Setiap klaim "berhasil" disertai perintah dari `11` dan keluarannya.
4. **Muat paling sedikit yang cukup.** Sebelum membuka berkas: "apakah rute INDEX menyuruh saya memuat ini?" `cat docs/*.md` adalah pelanggaran, bukan ketelitian.

4.3 Peta tujuh fase

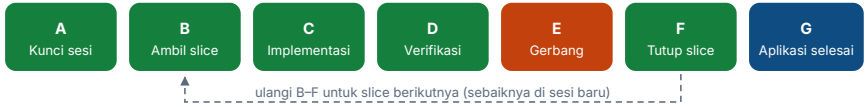


Diagram 4.1 — Fase A dijalankan sekali per sesi; B–F berulang per slice; G sekali di akhir.

4.4 Fase A: Muat Tier-0 dan kunci sesi

Buka sesi Claude Code baru di root proyek PRESQU.

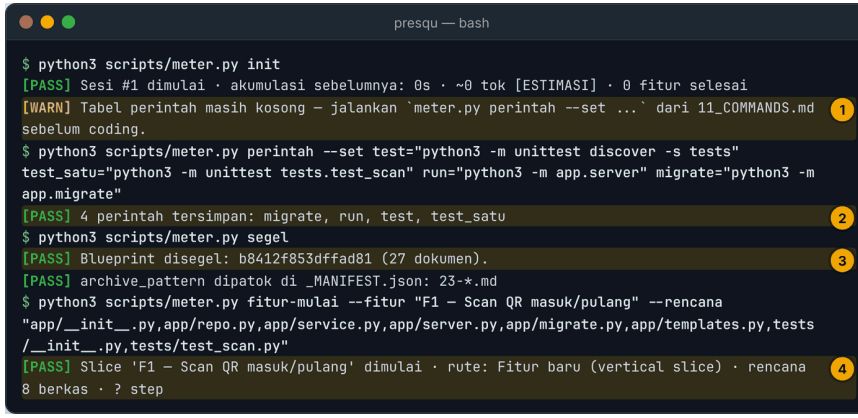
PROMPT

Mulai coding dari blueprint. Kerjakan slice pertama sesuai roadmap.

Skill membaca **hanya tiga berkas**: `CLAUDE.md`, `INDEX.md`, dan `docs/_MANIFEST.json`. Lalu ia memastikan kesehatan blueprint dengan `validate.sh`. Bila ada `[FAIL]`, coding tidak dimulai,

karena blueprint yang bertentangan akan melahirkan kode yang bertentangan.

Berikutnya, meteran sesi dinyalakan, perintah dipanen, dan blueprint disegel:



Gambar 4.1 — Membuka sesi: `init`, `perintah`, `segel`, dan `fitur-mulai`.

1. `init` langsung mengingatkan bahwa tabel perintah masih kosong. Agen tidak boleh menebak perintah.
2. `perintah --set` menyimpan perintah dari `11_COMMANDS.md` ke ledger. `11` adalah satu-satunya dokumen yang dibaca penuh di luar rute INDEX. Setelah ini, perintah selalu diambil dari ledger.
3. `segel` mengunci sidik jari dokumen blueprint. Bila nanti dokumen bergeser tanpa disengaja, skill akan tahu.
4. `fitur-mulai` mengunci rencana slice: delapan berkas yang boleh disentuh. Rute INDEX yang dipakai adalah "Fitur baru (vertical slice)".

4.5 Fase B: Ambil satu slice dan rencanakan

Satu slice saja. Jangan memborong dua fitur "karena mirip". Untuk F1, dokumen dimuat menurut rute INDEX (01, 02, 06, 07, 11, 23, plus 26 karena menyentuh tampilan). Dari `23`, agen membaca **hanya blok F1** dengan `grep -n` lalu `sed -n 'a,bp'`, bukan seluruh berkas.

Sebelum menulis kode, agen menyampaikan **Rencana Slice** kepada Anda:

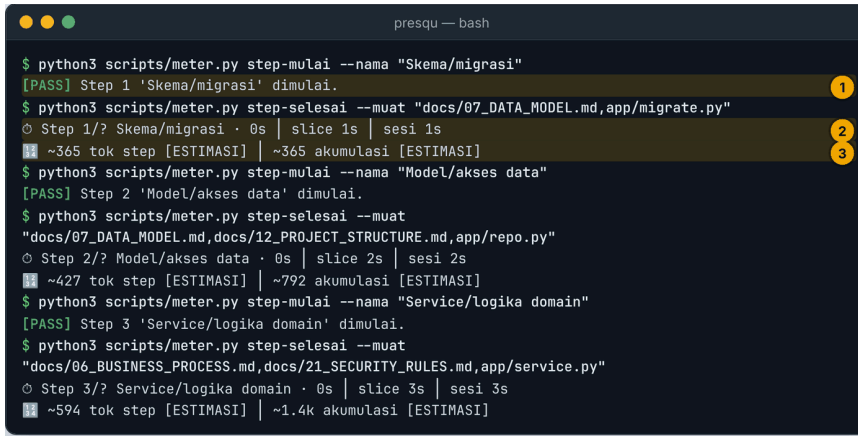
```
Rencana Slice — F1 Scan QR masuk/pulang
Kriteria   : docs/23 §F1 (5 butir)
Berkas     :
app/{__init__,migrate,repo,service,server,templates}.py,
            tests/{__init__,test_scan}.py
Step       : 1 skema · 2 repo · 3 service · 4 rute · 5 UI · 6 tes
· 7 gerbang
Verifikasi : python3 -m unittest tests.test_scan
Irreversibel: migrasi awal (tabel baru, tidak destruktif)
```

Gerbang scope. Fitur dicocokkan dulu ke `02_SCOPE.md`. Bila Anda meminta "sekalian ekspor PDF" dan itu ada di out-of-scope, agen berhenti dan bertanya. *Scope creep* paling sering masuk lewat kata "sekalian".

4.6 Fase C: Implementasi berlapis

Slice dikerjakan dalam urutan baku. Tiap lapis adalah satu step, dan setiap step ditutup dengan laporan counter satu baris.

Step	Lapis	Rumah aturan
1	Skema/migrasi	07 (+22 bila destruktif)
2	Model/akses data	07, 12
3	Service/logika domain	06, 12
4	Endpoint/rute	06, 08, 21
5	UI + empat state	26 (dilewati bila tanpa UI)
6	Tes	13
7	Gerbang fitur	23, 24

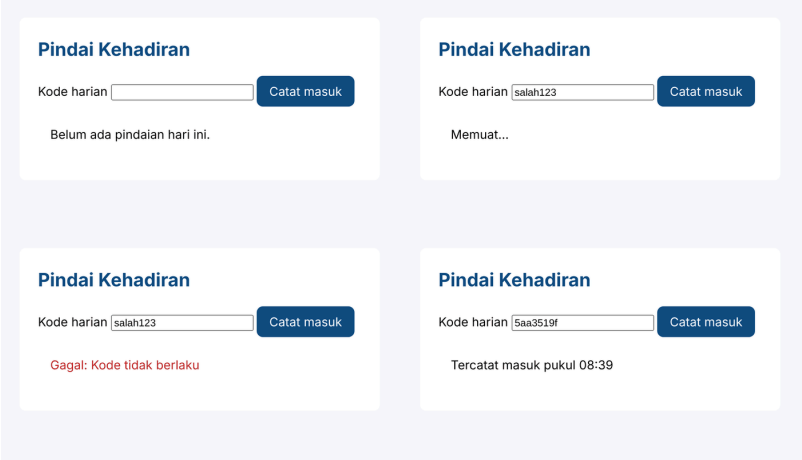


Gambar 4.2 — Tiga step pertama slice F1 beserta counter-nya.

1. `step-mulai` membuka step dengan nama lapisnya.
2. Baris waktu: durasi step, slice, dan sesi. Angka waktu bersifat **TERUKUR** dari jam dinding.
3. Baris token: dihitung dari ukuran berkas yang benar-benar dimuat (`--muat`) dibagi 3,5 karakter per token. Karena itu ia diberi label **[ESTIMASI]**.

> **Kejujuran angka.** Model tidak punya akses ke meteran token di sesi claude.ai. Angka `[ESTIMASI]` dihitung deterministik oleh skrip, bukan dikira-kira model. Angka `[TERUKUR]` hanya didapat lewat `token_ledger.py sync-cc` dari transkrip Claude Code. Keduanya tidak boleh dicampur.

Pada step 5, halaman `/scan` dibangun dengan token dari `26` dan empat state wajib. Berikut halaman aslinya setelah aplikasi dijalankan:



Gambar 4.3 — Empat state wajib halaman `/scan`: kosong (kiri atas), memuat (kanan atas), gagal (kiri bawah), sukses (kanan bawah).

Step 6 menulis tes untuk setiap alur kritikal di `13`, termasuk alur gagal dari `06`:

```
presqu — bash

$ python3 -m unittest discover -s tests -v
test_di_luar_jendela_422 (test_scan.TesPindai.test_di_luar_jendela_422) ... ok
test_kode_salah_422 (test_scan.TesPindai.test_kode_salah_422) ... ok
test_pindai_ganda_409 (test_scan.TesPindai.test_pindai_ganda_409) ... ok
test_pindai_sah_tercatat (test_scan.TesPindai.test_pindai_sah_tercatat) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.005s                                                  1

OK                                                                     2
```

Gambar 4.4 — Empat tes alur kritikal F1 lulus.

1. Empat tes: pindai sah, kode salah (422), di luar jendela waktu (422), dan pindai ganda (409).
2. Keluaran nyata ditempel apa adanya. Parafrase seperti "tes sudah lulus" tidak dihitung sebagai bukti.

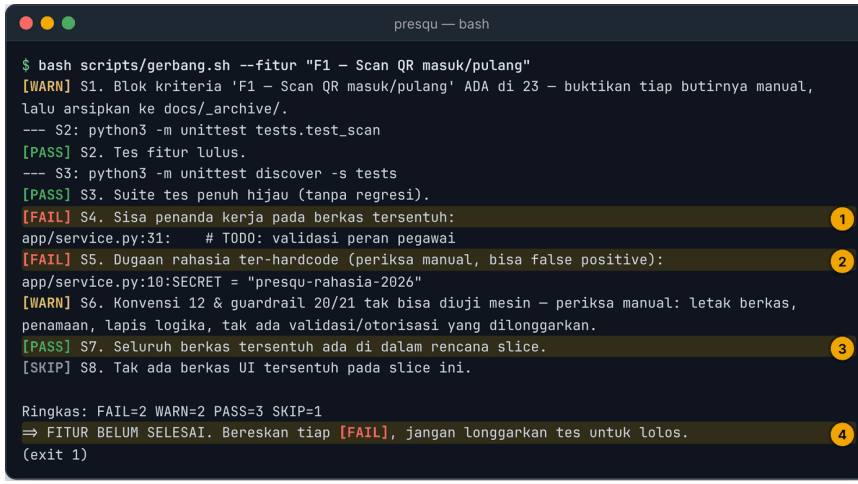
4.7 Fase D: Verifikasi

Blok verifikasi `23` dan suite tes dijalankan, lalu ekor keluarannya ditempelkan. Bila gagal, perbaikan mengikuti `18_REPAIR_RULES.md`: reproduksi, akar masalah, perbaikan

minimal, tes regresi. **Tiga kali gagal pada akar yang sama**, agen berhenti dan melapor: apa yang sudah dicoba, hipotesis akar masalah, dan dua opsi jalan keluar.

4.8 Fase E: Gerbang selesai fitur

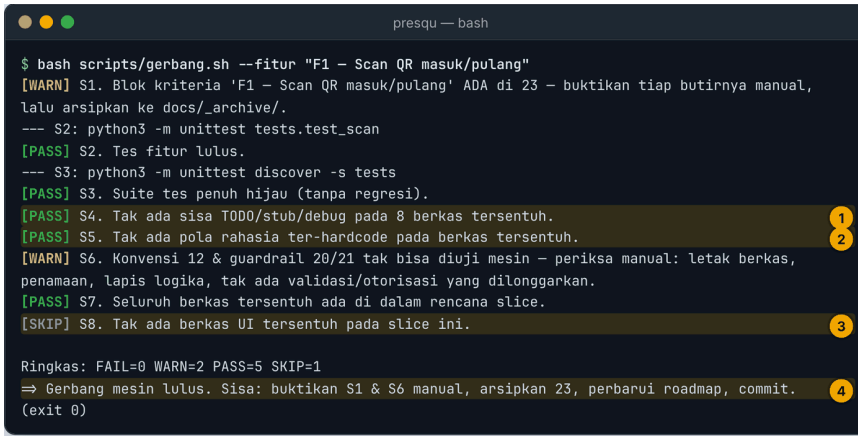
Inilah momen kebenaran. Jalankan gerbang dengan nama fitur:



Gambar 4.5 — Gerbang menolak slice F1 versi pertama.

1. **S4 gagal.** Ada komentar `TODO` tertinggal di `service.py`: validasi peran pegawai belum dikerjakan. Gerbang menolak penanda kerja pada berkas yang disentuh.
2. **S5 gagal.** Kunci rahasia ditulis langsung di kode, padahal `21` mewajibkan rahasia dari `PRESQU_SECRET`.
3. **S7 lulus.** Semua berkas yang disentuh ada di rencana slice. Tidak ada perubahan liar.
4. Kesimpulan tegas: **fitur belum selesai**, padahal tes S2 dan S3 hijau. Inilah alasan "tes lulus" tidak sama dengan "selesai".

Perbaikannya kecil dan tepat sasaran: rahasia dibaca dari variabel lingkungan, dan `TODO` diganti implementasi pemeriksaan peran yang sebenarnya (pegawai tak dikenal atau bukan peran "pegawai" mendapat 403). Gerbang dijalankan ulang:



Gambar 4.6 — Gerbang lulus setelah dua perbaikan.

1. S4 kini bersih pada kedelapan berkas.
2. S5 bersih: tidak ada pola rahasia.
3. S8 `[SKIP]`: templat UI PRESQU ditulis di berkas `.py`, sehingga tidak terdeteksi sebagai berkas UI. Ini batas nyata skrip. Empat state diperiksa manual (gambar sebelumnya), dan pengecualian ini **wajib dicatat** saat slice ditutup.
4. Gerbang mesin lulus. Sisa yang harus dibuktikan manusia: S1 (kriteria 23) dan S6 (konvensi dan guardrail).

Syarat	Yang diuji	Jenis
S1	Kriteria terima `23` terpenuhi	Manual
S2	Tes fitur (blok verifikasi) lulus	Mesin
S3	Suite penuh hijau, tanpa regresi	Mesin
S4	Tanpa TODO/FIXME/ `dd(` / `console.log` pada berkas tersentuh	Mesin
S5	Tanpa rahasia ter-hardcode; `.env` tidak ter-track	Mesin
S6	Konvensi `12`, guardrail `20` / `21`	Manual
S7	Berkas tersentuh ⊆ rencana slice	Mesin
S8	Empat state pada berkas UI	Mesin (heuristik)

> **Dilarang "hijau karena dihapus".** Melewatkan tes, melonggarkan asersi, menonaktifkan validasi atau otorisasi, atau mengubah skrip gerbang supaya lulus adalah kecurangan, bukan perbaikan. Bila gerbang hanya bisa lulus dengan cara itu, agen wajib berhenti dan bertanya.

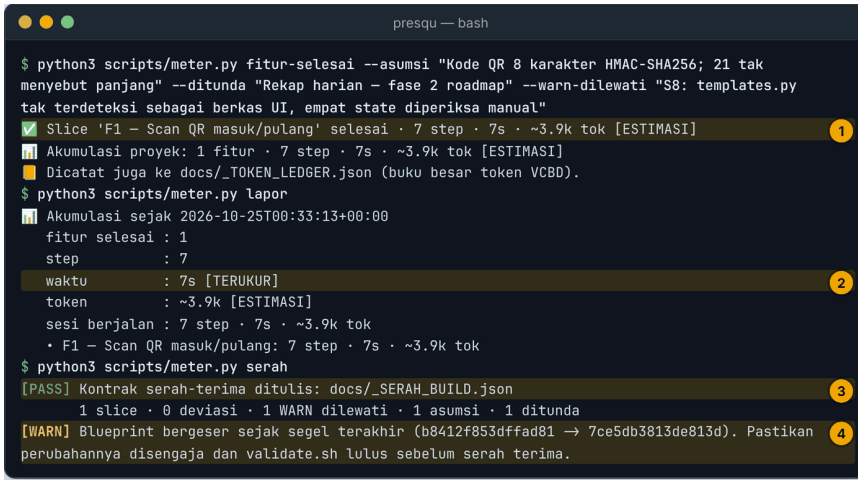
4.9 Fase F: Tutup slice

Setelah lulus, ada empat langkah penutup:

1. **Arsipkan blok 23** ke `docs/_archive/23-F1-scan-qr.md`. `23` hanya memuat fitur aktif, sehingga slice ke-10 tidak ikut membayar token fitur ke-1.
2. **Perbarui status** fase di `03_ROADMAP.md` (kolom status saja).
3. **Satu slice, satu commit** bermakna, sesuai `22`.
4. **Catat yang tak terlihat dari kode** dengan `fitur-selesai`.

```
$ python3 scripts/meter.py fitur-selesai \
    --asumsi "Kode QR 8 karakter HMAC-SHA256; 21 tak menyebut
panjang" \
    --ditunda "Rekap harian — fase 2 roadmap" \
    --warn-dilewati "S8: templates.py tak terdeteksi sebagai
berkas UI, empat state diperiksa manual"
```

Ruas-ruas ini bukan formalitas. Inilah satu-satunya jalan bagi auditor di Bab 5 untuk mengetahui utang yang sengaja diambil. Opsi lain yang tersedia: `--deviasi`, `--dokumen-diperbarui`, dan `--landmine`.



Gambar 4.7 — Menutup slice, melihat rekap, dan menerbitkan kontrak serah-terima.

1. Rekap slice: tujuh step, durasi, dan token estimasi. Angka juga dicatat ke ledger token VCBD.
2. `lapor` memisahkan waktu `[TERUKUR]` dan token `[ESTIMASI]`.
3. `serah` menulis `docs/_SERAH_BUILD.json`: satu slice, satu WARN dilewati, satu asumsi, satu pekerjaan ditunda.
4. Peringatan jujur: blueprint bergeser sejak disegel. Penyebabnya penutupan slice tadi (arsip 23 dan status roadmap), jadi pergeseran ini disengaja. Meski begitu, skrip tetap meminta Anda memastikannya.

> **Ganti sesi setelah tiap slice.** Sesi panjang adalah pembakar token terbesar. Blueprint dirancang agar sesi baru langsung produktif: *"Baca CLAUDE.md lalu INDEX.md, lanjutkan slice berikutnya dari 03_ROADMAP."*

4.10 Fase G: Kapan aplikasi boleh disebut selesai

Lima syarat, semuanya wajib benar:

1. semua fase `03_ROADMAP.md` berstatus selesai;
2. `23` tidak menyisakan fitur aktif;

3. suite tes penuh hijau pada konteks bersih;
4. `validate.sh` *exit* 0, artinya dokumen masih segaris dengan kode;
5. `25_RELEASE_CHECKLIST.md` dijalankan berurutan dan tiap langkah lulus.

Kurang satu pun, skill melaporkan "belum selesai" beserta sisanya. Setelah kelimanya benar, `meter.py serah` menerbitkan kontrak serah-terima final.

docs/_SERAH_BUILD.json

```
{
  "versi": "1.0",
  "sumber": "coding-vcbd 1.1",
  "diterbitkan": "2026-10-25T00:34:20+00:00",
  "blueprint": {
    "sidik_awal": "b8412f853dffad81",
    "sidik_saat_segel": "b8412f853dffad81",
    "sidik_saat_serah": "7ce5db3813de813d",
    "dokumen": 27,
    "bergeser": true,                                  (1)
    "bergeser_sejak_awal": true,
    "riwayat": []
  },
  "perintah": {                                        (2)
    "test": "python3 -m unittest discover -s tests",
    "test_satu": "python3 -m unittest tests.test_scan",
    "run": "python3 -m app.server",
    "migrate": "python3 -m app.migrate"
  },
  "ringkas": {
    "slice": 1,
    "step": 7,
    "detik": 7,
    "tok_est": 3930
```

Gambar 4.8 — Potongan `docs/_SERAH_BUILD.json`.

1. Sidik jari blueprint di awal, saat disegel, dan saat serah, beserta penanda pergeseran.
2. Perintah proyek ikut diwariskan, sehingga auditor tidak perlu menebak.

4.11 Diet token: aturan operasional

- Yang permanen hanya Tier-0: `CLAUDE.md` dan `INDEX.md`.

- Cari sebelum baca: `Glob` → `Grep -n` → `Read` rentang yang ditunjuk saja.
- Jangan menyalin isi dokumen ke jawaban chat; rujuk `docs/NN_NAME.md §bagian`.
- Bukti berupa diff dan ekor keluaran, bukan berkas kode utuh.
- Dokumen di daftar `collapsed` adalah stub; jangan dibuka berharap isi.

Latihan 4

1. Kerjakan slice F2 (rekap harian) PRESQU di sesi baru. Catat dokumen yang dimuat menurut rute INDEX.
2. Sengaja sentuh satu berkas di luar rencana slice. Apa yang dilaporkan S7?
3. Bandingkan angka token `[ESTIMASI]` dengan hasil `token_ledger.py sync-cc` di Claude Code.

Rangkuman Bab 4

- Fase A sekali per sesi; B–F per slice; G sekali di akhir.
- Rencana slice dikunci di muka, dan S7 menjaganya.
- Tes hijau belum tentu selesai; gerbang S1–S8 yang menentukan.
- Utang yang sengaja diambil dicatat lewat `fitur-selesai` dan diwariskan lewat `_SERAH_BUILD.json`.

BAB 5

review-vcbd: Audit, Uji, Lacak, Tambal

Kode yang lulus tes belum tentu kode yang dimaksud. Skill `review-vcbd` *menjawab satu pertanyaan:* ***di mana kode dan blueprint sudah tidak lagi bercerita hal yang sama?*** *Di bab ini kita mengaudit PRESQU, menemukan celah keamanan, melacak bug yang muncul sungguhan saat buku ini disusun, lalu menambal keduanya dengan tes regresi.*

Setelah membaca bab ini, Anda dapat:

- menjalankan lima mode: PINDAI, REVIEW, UJI, LACAK, TAMBAL;
- membaca dan mengelola Dosir Temuan berjenjang;
- memverifikasi temuan mesin, termasuk menolak *false positive*;
- menambal dengan tes regresi merah→hijau.

5.1 Empat Hukum Audit

1. **Selisih kode ↔ blueprint adalah TEMUAN,** bukan urusan yang dibereskan diam-diam. Skill melapor dan mengusulkan arah; keputusan (sesuaikan kode, atau perbarui dokumen) milik manusia.

2. **Tidak ada temuan tanpa bukti.** Setiap temuan membawa `path:baris` atau keluaran perintah. "Kemungkinan ada masalah di lapisan service" adalah firasat, bukan temuan.

3 **Tambalan tidak melebihi temuan.** Satu temuan = satu perbaikan minimal + satu tes regresi. Kode jelek di sebelahnya dicatat sebagai temuan baru, bukan dirapikan sekalian.

4 **Pindai lebar dengan skrip, baca dalam dengan model.** Skrip membaca 100% repo tanpa biaya token; model hanya membuka berkas yang ditunjuk temuan.

5.2 Lima mode

Mode	Untuk	Keluaran
PINDAI	Memotret seluruh repo (selalu langkah pertama)	`docs/_TEMUAN.json` + ringkasan
REVIEW	Kesesuaian kode ↔ blueprint pada delapan sumbu	Temuan berjenjang + usulan arah
UJI	Menjalankan dan melengkapi pengujian, termasuk uji asap HTTP	Hasil suite, celah tes
LACAK	Menemukan akar masalah satu gejala	Akar masalah + bukti reproduksi
TAMBAL	Memperbaiki temuan yang disetujui	Patch minimal + tes regresi + gerbang

5.3 Skenario: kode yang masuk lewat pintu samping

Bayangkan beberapa hari setelah slice F1 ditutup, seseorang menambahkan fungsi pencarian pegawai ke `app/repo.py` di luar rencana slice mana pun:

```
def cari_nama(conn, nama):
    return conn.execute("SELECT id, name FROM employees WHERE name
LIKE '%"
                        + nama + "%'").fetchall()
```

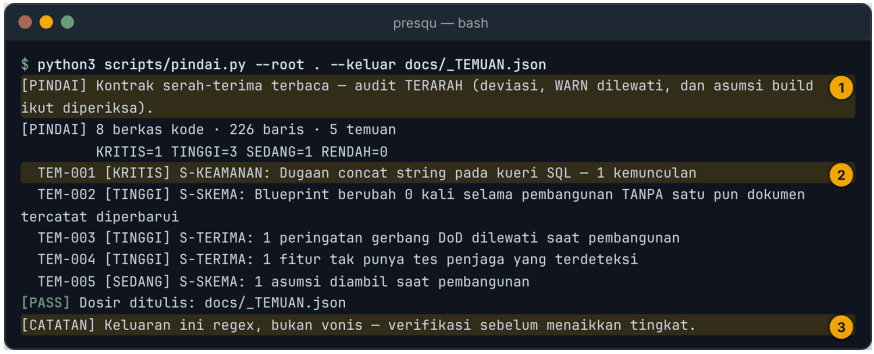
Kodenya berjalan. Tidak ada tes yang merah. Tetapi `21_SECURITY_RULES.md` mewajibkan kueri terparameter. Mari lihat apakah audit menangkapnya.

5.4 Mode PINDAI

PROMPT

Audit kode PRESQU terhadap blueprint. Mulai dari pemindaian penuh.

Skill memastikan ini paket VCBD, membaca Tier-0, menjalankan `validate.sh` (blueprint yang bengkok akan menghasilkan temuan palsu), lalu membaca `_SERAH_BUILD.json` bila ada. Setelah itu, pemindaian penuh:



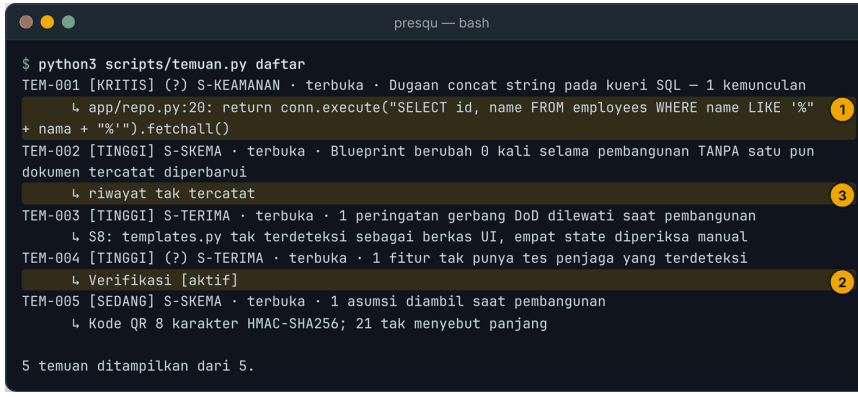
Gambar 5.1 — Pemindaian penuh PRESQU dalam mode audit terarah.

1. Kontrak serah-terima dari Bab 4 terbaca, sehingga audit berjalan **terarah**: deviasi, WARN yang dilewati, dan asumsi diperiksa lebih dulu.
2. Temuan KRITIS: dugaan penggabungan string pada kueri SQL. Fungsi pintu samping tadi tertangkap.
3. Pengingat penting dari skrip itu sendiri: keluaran ini regex, bukan vonis.

Pada tahap PINDAI, skill **tidak membuka berkas apa pun**. Ia hanya menyampaikan ringkasan: mode audit, jumlah berkas dan baris, temuan per tingkat, dan lima temuan teratas.

5.5 Mode REVIEW: verifikasi temuan

Sekarang model bekerja. Ia membuka **hanya** berkas yang ditunjuk temuan, memverifikasi, lalu menaikkan, menurunkan, atau menolak.



Gambar 5.2 — Daftar temuan beserta buktinya.

1. **TEM-001** menunjuk `app/repo.py:20`. Setelah dibuka, terbukti: masukan pengguna digabung langsung ke SQL. Temuan ini **sah**.
2. **TEM-004** mengira "Verifikasi" adalah nama fitur, padahal itu sub-judul di dalam blok F1 di `23`. Tes penjaganya ada di `tests/test_scan.py`. Ini ***false positive***.
3. **TEM-002** menyatakan blueprint bergeser tanpa satu pun dokumen tercatat diperbarui. Ini temuan sah tentang *pencatatan*: saat menutup slice F1 kita mengubah `03` dan `23`, tetapi tidak mengisi `--dokumen-diperbarui`. Kalimatnya menyebut "0 kali" karena riwayat pergeseran tidak tercatat.

Temuan dikelola lewat `temuan.py`. Yang palsu ditolak dengan alasan tertulis; yang sah disetujui untuk ditambal.



Gambar 5.3 — Menolak *false positive* dan menyetujui temuan yang akan ditambal.

1. TEM-004 ditolak, dengan alasan yang tersimpan di dosir.
2. TEM-001 disetujui. Hanya temuan yang disetujui yang boleh ditambal.

Delapan sumbu REVIEW

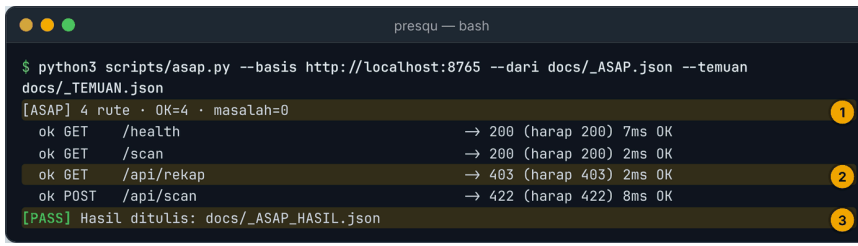
Sumbu	Patokan	Contoh temuan
Skema	07	Tabel/kolom di kode tak ada di 07, atau sebaliknya
Struktur & penamaan	12	Logika bisnis di controller; SQL di luar `repo.py`
Keamanan	20, 21	Concat SQL, rahasia ter-hardcode, otorisasi dilewati
Peran & akses	05	Endpoint tanpa pemeriksaan peran yang dijanjikan
Alur proses	06	Hanya *happy path*; cabang gagal tak ada
Kriteria terima	23 + `_archive/`	Fitur diterima tanpa tes penjaga
UI	26	Hex/px di luar tabel token; state wajib kurang
Scope & stack	02, 09	Fitur out-of-scope terlanjur dibangun; dependensi asing

Empat tingkat temuan. KRITIS (rusak, tidak aman, melanggar kriteria terima) dan **TINGGI** (menyimpang pada sumbu berdampak) wajib nol sebelum rilis. **SEDANG** boleh ditunda dengan alasan tertulis. **RENDAH** cukup dicatat.

5.6 Mode UJI

Tiga lapis, dari yang termurah: (1) suite yang sudah ada, (2) celah tes, yaitu alur kritikal di `13` yang belum punya tes, dan (3) uji dinamis, yaitu aplikasi benar-benar dijalankan. Untuk lapis ketiga, daftar rute **diturunkan** dari `06` dan `23`, bukan dikarang:

```
{"rute": [
  {"path": "/health",    "harap": 200},
  {"path": "/scan",      "harap": 200, "catatan": "06 alur F1"},
  {"path": "/api/rekap", "harap": 403, "catatan": "05: tanpa peran
admin"},
  {"path": "/api/scan",  "metode": "POST", "harap": 422,
"catatan": "badan kosong"}
]}
```



Gambar 5.4 — Uji asap empat rute PRESQU yang sedang berjalan.

1. Keempat rute memberi status sesuai harapan.
2. Rute rekap menolak permintaan tanpa peran admin dengan 403. Ini bukti otorisasi ditegakkan di server, sesuai matriks `05`.
3. Hasil disimpan; dengan `--temuan`, masalah langsung disuntikkan ke dosir.

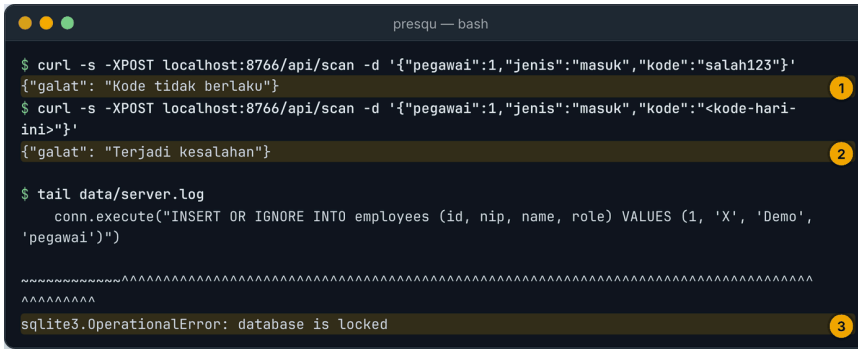
`asap.py` juga memeriksa kebocoran jejak galat (*stack trace*) di badan respons.

5.7 Mode LACAK: sebuah bug sungguhan

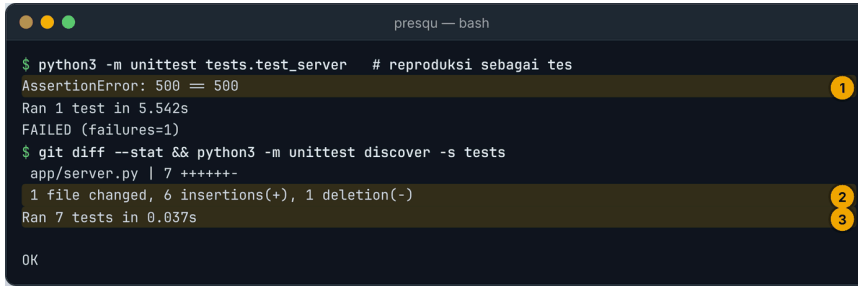
Saat tangkapan layar Bab 4 diambil, aplikasi PRESQU sempat menolak pindaian yang sah dengan pesan "Terjadi kesalahan". Bug ini tidak direkayasa untuk buku; ia muncul sendiri. Mari

lacak dengan protokol yang benar.

Langkah 1: reproduksi dulu


presqu — bash
$ curl -s -XPOST localhost:8766/api/scan -d '{"pegawai":1,"jenis":"masuk","kode":"salah123"}'
{"galat": "Kode tidak berlaku"}
1
$ curl -s -XPOST localhost:8766/api/scan -d '{"pegawai":1,"jenis":"masuk","kode":"<kode-hari-ini>"}'
{"galat": "Terjadi kesalahan"}
2
$ tail data/server.log
conn.execute("INSERT OR IGNORE INTO employees (id, nip, name, role) VALUES (1, 'X', 'Demo', 'pegawai')")
~~~~~~~~~~~~^^^
~~~~~~~~~~~~

Langkah 4: tes yang gagal, lalu tambalan minimal



Gambar 5.6 — Tes regresi merah, tambalan enam baris, lalu suite penuh hijau.

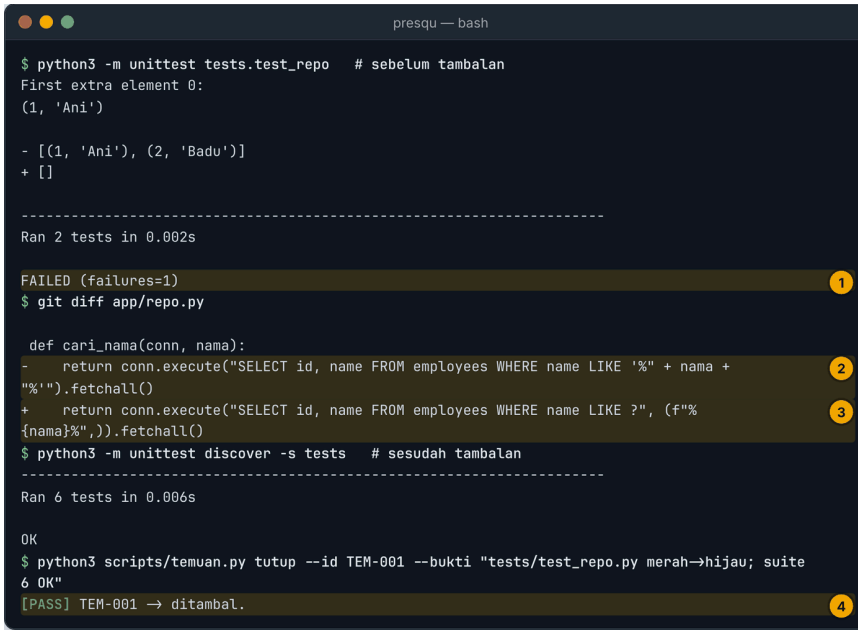
1. Gejala dibekukan menjadi tes: dua pindaian berturut-turut tidak boleh menghasilkan 500. Tes ini merah sebelum tambalan.
2. Tambalan hanya menyentuh `server.py`: `commit` setelah baris awal, dan koneksi ditutup di blok `finally`.
3. Tujuh tes lulus, termasuk tes regresi yang baru.

> **Tiga kali gagal, berhenti.** Bila tiga percobaan pada akar yang sama gagal, skill berhenti dan melapor: apa yang sudah dicoba, hipotesis tersisa, dan dua opsi jalan keluar. Hipotesis yang dilabeli fakta adalah cara tercepat menambal hal yang salah.

5.8 Mode TAMBAL: menutup TEM-001

Kembali ke celah SQL. Aturannya: hanya temuan yang disetujui, perbaikan minimal, dan tes regresi yang **gagal sebelum tambalan dan lulus sesudahnya**.

```
def test_masukan_jahat_tidak_membocorkan_semua_baris(self):
    self.assertEqual(repo.cari_nama(self.conn, "%' OR 1=1 --"),
[])
```



Gambar 5.7 — Siklus TAMBAL lengkap untuk TEM-001.

1. Sebelum tambalan, tes gagal: masukan jahat membocorkan *seluruh* pegawai. Celahnya terbukti, bukan diduga.
2. Baris lama yang menggabungkan string ke SQL.
3. Baris baru dengan placeholder `?`. Hanya satu baris yang berubah.
4. Temuan ditutup dengan bukti berupa perintah dan hasilnya.

> **Sebuah pelajaran kecil.** Versi pertama tes regresi untuk buku ini memakai masukan `x' OR '1'='1`, dan tes itu ternyata *lulus* pada kode yang rentan, karena sisa pola `LIKE` membuat kondisinya tetap salah. Tes regresi yang tidak pernah merah tidak membuktikan apa pun. Itulah sebabnya protokol mewajibkan melihat tes gagal *sebelum* menambal.

Setelah tambalan, gerbang milik `coding-vcbd` dijalankan ulang untuk fitur terdampak. Bila tambalan mengubah fakta milik dokumen, hanya dokumen pemiliknya yang diperbarui, lalu `validate.sh`.

5.9 Laporan dan syarat sehat

docs/_LAPORAN_TEMUAN.md

```
# Laporan Temuan — review-vcbd

Dipindai: 2026-10-25T00:34:27+00:00 · Dilaporkan: 2026-10-25T00:35:08+00:00

Cakupan: 8 berkas kode · 226 baris — dipindai seluruhnya oleh `pindai.py`.

## Status kesehatan

- Temuan terbuka: **3** (KRITIS/TINGGI: **2**)
- Sudah ditambal: 1
- Ditolak (false positive): 1

> **BELUM SEHAT** — masih ada temuan KRITIS/TINGGI terbuka; rilis ditahan.   (1)

## TINGGI (2)

### TEM-002 — Blueprint berubah 0 kali selama pembangunan TANPA satu pun dokumen tercatat diperbarui   (2)

- Sumbu: S-SKEMA · Patokan: `docs/_SERAH_BUILD.json`
- Bukti:
  - `riwayat tak tercatat`
- Usul: Cocokkan tiap pergeseran dengan daftar dokumen_diperbarui; pergeseran tanpa catatan berarti fakta berubah diam-diam.

### TEM-003 — 1 peringatan gerbang DoD dilewati saat pembangunan
```

Gambar 5.8 — Laporan temuan setelah satu tambalan dan satu penolakan.

1. Status tegas: **belum sehat**, karena masih ada temuan TINGGI terbuka. Rilis ditahan.
2. TEM-002 menunggu keputusan: catat dokumen yang diperbarui di kontrak serah-terima, lalu tutup temuannya.

Aplikasi dinyatakan **sehat** bila kelima syarat ini terpenuhi berurutan: (1) tidak ada temuan KRITIS/TINGGI terbuka, (2) suite penuh hijau pada konteks bersih, (3) tiap fitur di 23 dan arsipnya punya tes penjaga, (4) `validate.sh` *exit* 0, (5) checklist rilis lulus.

5.10 Anti-pattern audit

- Membaca berkas satu per satu "untuk memahami aplikasi" tanpa PINDAI lebih dulu.
- Menganggap keluaran `pindai.py` sebagai vonis final.

- Mengubah dokumen agar cocok dengan kode yang salah tanpa keputusan manusia. Itu memalsukan patokan.
- Menyatakan aplikasi teruji padahal tidak ada satu pun rute nyata yang disentuh.
- Menutup temuan dengan melonggarkan tes.

Latihan 5

1. Tambahkan satu warna hex di luar tabel token ke templat PRESQU. Sumbu mana yang seharusnya menangkapnya?
2. Tutup TEM-002 dengan cara yang benar: catat dokumen yang diperbarui, lalu jalankan PINDAI ulang.
3. Tulis entri `_ASAP.json` untuk rute rekap dengan peran admin.

Rangkuman Bab 5

- PINDAI selalu pertama; model hanya membuka berkas yang ditunjuk temuan.
- Temuan mesin diverifikasi; *false positive* ditolak dengan alasan tertulis.
- LACAK dimulai dari reproduksi dan `16_DEBUGGING_GUIDE`.
- TAMBAL selalu dengan tes regresi yang terbukti merah lebih dulu.

BAB 6

reverse-vcbd: Membedah Kode Warisan

Kebanyakan aplikasi yang kita warisi tidak datang dengan gambar kerja. Skill `reverse-vcbd` *mengubah repo yang sudah berjalan menjadi dosir bukti, lalu menjadi paket blueprint yang sah dipakai agen coding, dengan syarat mutlak: setiap fakta bisa ditunjuk asalnya.*

Setelah membaca bab ini, Anda dapat:

- menjalankan pemindaian statis dan pemindaian data *read-only*;
- membedakan apa yang *mungkin* (dari kode) dan apa yang *terjadi* (dari data);
- membaca derajat bukti `[KODE]`, `[DATA]`, `[USULAN]`, `[ISI:]`;
- merakit manifest dan kerangka blueprint dari dosir.

6.1 Mengapa butuh alat khusus?

`vcbd` sudah mewajibkan *grounding* untuk proyek brownfield: skema, perintah, struktur, dan stack harus **diekstrak dari sumber aktual**, bukan ditanyakan. Tetapi mandat tanpa alat berakhir sama: model menebak lalu menuliskannya dengan nada pasti, dan `coding-vcbd` di hilir memperlakukan tebakan itu sebagai kebenaran. `reverse-vcbd` memasok lapisan buktinya.

6.2 Lima Hukum

1. **Tidak ada klaim tanpa bukti yang bisa ditunjuk.** Tiap fakta membawa `path:baris` atau `[DATA: kueri]`. Yang tak berbukti ditulis `[USULAN]` atau `[ISI:]`.

2. **Kode dan data menang atas ingatan; niat tetap milik manusia.** Mengapa aplikasi ini ada dan apa yang mau dipertahankan hanya manusia yang tahu, dan itu ditanyakan.

3. **Kode bicara soal yang MUNGKIN; data bicara soal yang TERJADI.** Tanpa bukti data, dilarang menyatakan sebuah jalur mati atau fitur tak terpakai.

4. **Cakupan dilaporkan apa adanya.** Blueprint yang mengaku lengkap padahal baru 60% adalah kegagalan termahal skill ini.

5. **Rahasia dan data pribadi tidak pernah masuk dokumen.** Penegakannya ada di `redaksi.py`, bukan pada kedisiplinan model.

6.3 Tiga mode

Mode	Keluaran	Pakai ketika
RECON	`docs/_RECON/dosir-recon.json` + `laporan-cakupan.md`	Hanya ingin tahu isi perut aplikasi
PETA	`docs/06_BUSINESS_PROCESS.md` lepas	Hanya butuh peta alur proses
BLUEPRINT	Paket VCBD lengkap	Aplikasi akan dilanjutkan, ditulis ulang, atau diserahkan ke agen

Mode PETA dan BLUEPRINT sama-sama menjalankan RECON lebih dulu. Tidak ada jalan pintas melewati pengumpulan bukti.

6.4 Proyek contoh: SI-CUTI

SI-CUTI adalah aplikasi pengajuan cuti bergaya Laravel 10: tujuh rute, tiga tabel migrasi, dua model, satu controller, satu jadwal cron, satu *queue job*, dan satu integrasi keluar ke SIMPEG. Ada juga berkas `.env` berisi sandi basis data, serta basis data SQLite berisi 40 pegawai dan 312 pengajuan.

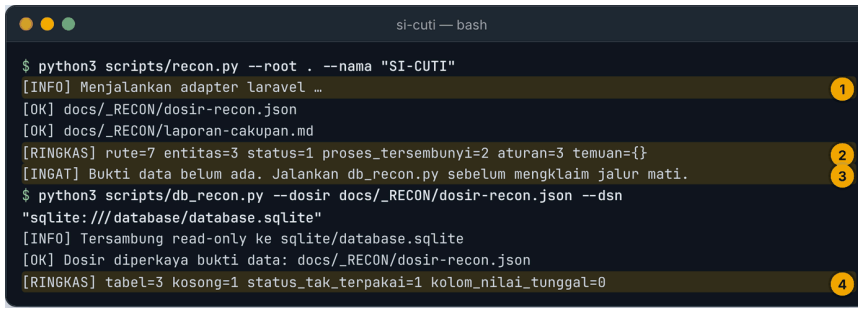
PROMPT

Buatkan blueprint dari kode SI-CUTI yang sudah ada. Basis data boleh dibaca read-only.

6.5 Fase 0: Triase dan izin

Skill mendeteksi profil (Laravel, PHP native, Flutter, atau campuran; campuran adalah hal normal), lalu **meminta izin basis data secara eksplisit**, dengan sifat yang ditegaskan: read-only, agregat saja, kredensial tidak disimpan. Bila ditolak, skill tetap jalan, tetapi semua klaim "jalur mati" terkunci oleh Hukum 3. Bila `docs/` atau `CLAUDE.md` sudah ada, skill menawarkan cadangan `.bak`, mode merge, atau subfolder. Ia tidak menimpa diam-diam.

6.6 Fase 1 dan 2: Pindai statis dan pindai data



Gambar 6.1 — Pemindaian statis lalu pemindaian data SI-CUTI.

1. Adapter Laravel terpilih otomatis dari `composer.json`.
2. Ringkasan mesin: 7 rute, 3 entitas, 1 mesin status, 2 proses tak kasat mata (cron dan *queue job*), 3 aturan validasi.
3. Pengingat Hukum 3: tanpa bukti data, jalur mati belum boleh diklaim.
4. Setelah membaca basis data: satu tabel kosong dan satu nilai status yang tidak pernah terjadi.

> **Jangan membaca repo manual dulu.** Baca dosirnya. Membaca ratusan berkas sendiri membakar token untuk pekerjaan yang sudah deterministik, dan hasilnya justru kurang lengkap.

Laporan cakupan ditulis untuk manusia:

docs/_RECON/laporan-cakupan.md

```
# Laporan Cakupan Recon — SI-CUTI

Dibuat: 2026-10-25T00:35:38 · Profil terdeteksi: laravel

## Cakupan (angka apa adanya, tidak dibulatkan ke atas)

| Ukuran | Nilai |
|---|---|
| Berkas relevan di repo | 8 |
| Berkas terpindai | 8 (100.0%) |                                  (1)
| Rute ditemukan | 7 |
| Rute dengan penjaga akses terbaca | 6 |
| Entitas/tabel ditemukan | 3 |
| Mesin status ditemukan | 1 |
| Bukti data (basis data) | BELUM — klaim jalur mati tidak sah |  (2)

**Tabel tanpa rute terkait (3):** pegawais, pengajuans, cuti_lama

**Berkas sengaja TIDAK dibaca (berisi kredensial):** .env                  (3)

## Pertanyaan terbuka
```

Gambar 6.2 — Laporan cakupan SI-CUTI dengan angka apa adanya.

1. Persentase berkas terpindai ditulis tanpa pembulatan.
2. Status bukti data ditulis jujur. Laporan ini dibuat oleh `recon.py` sebelum `db_recon.py` berjalan; bukti data masuk ke dosir JSON.
3. `.env` sengaja tidak dibaca karena berisi kredensial. Keberadaannya dicatat, isinya tidak. Pemeriksaan setelah demo: kata sandi di `.env` tidak muncul di satu pun berkas keluaran.

6.7 Kode vs data: siklus status pengajuan

Inilah contoh terbaik Hukum 3. Potongan dosir berikut memperlihatkan mesin status kolom `pengajuans.status`:

docs/_RECON/dosir-recon.json (mesin_status)

```
{
  "entitas": "pengajuans",
  "kolom": "status",
  "nilai": [
    "diajukan",
    "disetujui",
    "ditolak",
    "dibatalkan"                                                    (1)
  ],
  "sumber_nilai": "enum migrasi",
  "transisi": [
    {
      "dari": "[ISI: asal]",
      "ke": "disetujui",
      "pemicu": "$p->status = 'disetujui';",
      "bukti": "app/Http/Controllers/CutiController.php:14",
      "derajat": "KODE"                                             (2)
    },
    {
      "dari": "[ISI: asal]",
      "ke": "ditolak",
      "pemicu": "$p->status = 'ditolak';",
      "bukti": "app/Http/Controllers/CutiController.php:20",
      "derajat": "KODE"
    }
  ],
  "nilai_teramati": [
    "diajukan",
    "disetujui",
    "ditolak"
  ],
  "bukti": "database/migrations/2023_01_01_000002_create_pengajuans_table.php:10",
  "nilai_tak_terpakai": [                                           (3)
    "dibatalkan"
  ]
}
```

Gambar 6.3 — Mesin status di dosir: gabungan bukti kode dan bukti data.

(1) Migrasi mendefinisikan empat nilai, termasuk `dibatalkan`. Ini yang *mungkin*.

(2) Transisi ke `disetujui` dan `ditolak` ditemukan di controller, lengkap dengan `path:baris` dan derajat `KODE`. Status asalnya `[ISI: asal]`, karena kode tidak menyebutkannya.

(3) Data membuktikan `dibatalkan` **tidak pernah terjadi** di 312 baris. Ini yang *terjadi*.

Temuan seperti ini tidak langsung dianggap "fitur mati". Di Fase 3, skill bertanya kepada manusia: **dibuang, dipertahankan, atau dipakai kanal lain?**

6.8 Fase 3 dan 4: Wawancara kilat dan gerbang Ringkasan Temuan

Model merakit alur proses dari bahan dosir: siklus status, rute, penjaga akses, dan proses terjadwal. Urutan yang tidak berbukti tetap berlabel `[USULAN]`. Lalu wawancara kilat, **hanya** untuk hal yang tak terbaca mesin: tujuan aplikasi, batas perubahan, apa yang dipertahankan atau dibuang, roadmap, dan larangan teknologi. Skema, rute, stack, dan perintah dilarang ditanyakan, karena semuanya sudah ada di dosir.

Fase 4 adalah gerbang keras. Bedanya dengan `vcbd`: yang dikonfirmasi bukan kebutuhan yang digali, melainkan **kondisi eksisting yang ditemukan**.

Contoh ringkasan yang disusun dari dosir SI-CUTI:

```
Ringkasan Temuan — SI-CUTI
Cakupan   : 8/8 berkas (100%), bukti data: ya (read-only, agregat)
Profil    : Laravel 10
Rute 7 · Entitas 3 · Alur 3 · Proses tersembunyi 2 · Integrasi 1
Berisiko  : /lama/ekspor tanpa penjaga akses; tabel cuti_lama
kosong;
            status 'dibatalkan' tak pernah terjadi
[ISI:]    : 15 slot menunggu jawaban manusia

Konfirmasi: rakit paket blueprint dari temuan di atas?
Jawab 'ya' atau koreksi bagian yang salah.
```

6.9 Fase 5: Generasi dan gerbang mesin



Gambar 6.4 — Dosir dirakit menjadi `_MANIFEST.json`.

1. Lima belas slot `[ISI:]` dihitung terang-terangan. Itulah bagian yang hanya bisa diisi manusia.

Setelah manifest, `scaffold.py` **milik** `vcbd` menulis kerangka dan `INDEX.md`. `reverse-vcbd` tidak pernah menulis format sendiri. Satu penulis format berarti satu format.

```
$ python3 ~/.claude/skills/vcbd/scripts/scaffold.py --root .
$ bash scripts/validate.sh        # bereskan tiap [FAIL]
```

6.10 Derajat bukti

Label	Artinya	Contoh
`[KODE: path:baris]`	Terbaca di berkas, bisa diperiksa ulang siapa pun	`[KODE: routes/web.php:7]`
`[DATA: kueri]`	Terbukti dari agregat basis data	`[DATA: GROUP BY status pada pengajuans]`
`[USULAN]`	Rekonstruksi penalaran dari bahan berbukti	Urutan langkah yang belum terbukti
`[ISI:]`	Hanya manusia yang tahu; sengaja dikosongkan	Alasan aturan, rencana ke depan

Label `[TERVALIDASI]` sengaja tidak dipakai: label itu tidak memberi tahu pembaca *di mana* harus memeriksa ulang.

6.11 Fase 6: Serah terima

Skill menyampaikan lokasi dosir dan laporan, persen cakupan apa adanya, daftar `[ISI:]` yang menunggu, dan tiga temuan paling berisiko. Satu pengingat penting: **dokumen ini merekam keadaan SEKARANG, bukan keadaan yang diinginkan.** Keputusan mengubahnya adalah pekerjaan berikutnya, lewat Mode Pembaruan `vcbd`.

> **Jangan "memperbaiki" sambil mendokumentasikan.** Mengganti nama tabel yang jelek atau merapikan alur yang aneh saat menulis `07` adalah anti-pattern. Usulan perbaikan hidup di `open_questions`, bukan diselundupkan ke dokumen.

Rantai yang disarankan untuk aplikasi warisan: `reverse-vcbd` → `review-vcbd` (mode PINDAI) → `coding-vcbd`.

Latihan 6

1. Jalankan `recon.py` pada salah satu repo lama Anda. Berapa persen cakupannya?
2. Temukan satu nilai status yang hanya ada di kode. Apa jawaban pemilik aplikasi: dibuang, dipertahankan, atau dipakai kanal lain?
3. Cari di keluaran dosir apakah ada nilai dari `.env` yang bocor. Seharusnya tidak ada.

Rangkuman Bab 6

- RECON selalu lebih dulu; dosir adalah buku bukti mesin.
- Kode menunjukkan yang mungkin; data menunjukkan yang terjadi.
- Gerbang Ringkasan Temuan mengonfirmasi kondisi eksisting sebelum dokumen ditulis.
- Format blueprint tetap ditulis oleh `scaffold.py` milik `vcbd`.

BAB 7

reverse-web-vcbd: Membedah Kotak Hitam

Banyak aplikasi instansi hanya bisa dibuka lewat peramban: buatan vendor yang sudah pergi, kodenya tidak diserahkan, atau servernya milik pihak lain. `reverse-web-vcbd` *membedahnya dari luar, cukup dari rekaman lalu lintas peramban, lalu berhenti di dosir. Penulisan blueprint tetap tugas* `vcbd`*.*

Setelah membaca bab ini, Anda dapat:

- menangkap HAR per peran secara etis dan efektif;
- membedah semua bahan dengan satu perintah `jalankan.py`;
- membaca matriks akses dengan benar ("tak teramati" ≠ "dilarang");
- menyerahkan draf manifest ke `vcbd`.

7.1 Lima Hukum

1. **Tak ada klaim tanpa bukti.** Label `[AMATI: har:berkas#n]`, `[AMATI-KORELASI]`, `[AMATI-JS]`, `[AMATI-URUTAN]`, `[INFER]`, `[USULAN]`, `[ISI:]`. Label tidak pernah dinaikkan hanya karena kalimatnya terdengar masuk akal.

2. **Kotak hitam punya batas, dan batasnya ditulis.** Skema basis data fisik, aturan server yang tak terpicu, proses latar, dan integrasi server-ke-server tidak terlihat dari peramban. Semuanya `[ISI:]`.

3 **"Tak teramati" ≠ "tidak ada" ≠ "dilarang".** Larangan hanya disimpulkan dari 401/403 nyata.

4 **Mesin membaca HAR, model membaca laporan.** HAR bisa puluhan MB; membacanya manual membakar token dan hasilnya lebih buruk.

5 **Pasif, berwenang, dan bersih data pribadi.** Tanpa fuzzing, tanpa tebak URL, tanpa aksi destruktif. Nilai rahasia dan PII disaring `redaksi.py`.

> **Batas etis.** Gunakan hanya pada aplikasi yang Anda berwenang telaah: milik instansi Anda, buatan Anda sendiri, atau milik vendor yang kontraknya mengizinkan. Temuan keamanan di bab ini adalah efek samping pengamatan pasif (header, bendera cookie), bukan hasil uji penetrasi, dan tidak boleh "dibuktikan" dengan mencoba celahnya.

7.2 Menangkap HAR per peran

Kualitas dosir hampir seluruhnya ditentukan saat menangkap bahan. Aturannya: **satu HAR = satu peran = satu sesi**.

Langkah 1 Buka jendela Incognito baru. Tekan `F12`, buka tab **Network**, lalu centang **Preserve log** dan **Disable cache**.

Langkah 2 Login dengan akun peran pertama, lalu jalankan rencana jelajah (15–30 menit): klik setiap menu sekali; di tiap tabel buka satu detail; buka setiap formulir; kirim formulir kosong **satu kali** untuk menangkap bentuk galat validasi.

Langkah 3 Jalankan **satu alur utuh pada objek yang sama**: ajukan (peran A) → buka daftar → logout → setujui (peran B) → buka daftar lagi. Dari sinilah transisi status `[AMATI-KORELASI]` terbaca.

Langkah 4 Klik kanan di daftar permintaan → **Save all as HAR with content**. Simpan sebagai `<peran>.har`, logout, ulangi untuk peran berikutnya.

> **Jangan klik tombol hapus atau kirim-final pada data produksi.** Skrip menangkap label tombol dari HTML tanpa perlu diklik. Bila HAR tidak memungkinkan, jalur cadangannya adalah halaman tersimpan (`Ctrl` + `S`, *HTML only*), satu folder per peran.

7.3 Bedah dan rakit dengan satu perintah

Untuk demo ini, bahan diambil dari *fixture* uji bawaan skill: HAR peran pegawai dan admin dari aplikasi cuti bergaya Laravel, plus satu halaman tersimpan. *Fixture* ini sengaja ditanami NIP, nama, email, sandi, dan token agar penyaringan data pribadinya bisa dibuktikan.

```
si-cuti-web — bash

$ python3 scripts/jalankan.py --nama "SI-CUTI Web" --root proyek pegawai=pegawai.har
admin=admin.har pegawai=simpan/
  · pegawai      pegawai.har              14 entri → 11 endpoint                    (1)
  · admin        admin.har                6 entri → 5 endpoint
  · pegawai      simpan                   1 halaman
[ok] SI-CUTI Web · peran 2 (admin, pegawai)
    halaman 8/11 (73%) · endpoint 14 (+0 dari JS) · form 4/5                       (2)
    modul 4 · entitas 2 · enum 4 · transisi 1 · alur lintas peran 2
    stack: Laravel, Laravel/Angular XSRF · pola Hibrida (halaman server + AJAX/JSON)
    ⚠ [AMATI] `GET /cuti/{id}/hapus` mengubah data lewat GET — jangan pernah di-  (3)
prefetch/crawl; bukti har:pegawai.har#11
    ⚠ [INFER] `POST /cuti/{id}/setujui` memakai envelope JSON sendiri (success/status/error) —
galat bisa datang dengan HTTP 200
    ⚠ [AMATI] cookie sesi `XSRF-TOKEN` tanpa Secure
    belum: 3 halaman, 1 form                                                       (4)
    → proyek/docs/_RECON_WEB/{dosir-web.json, laporan-cakupan.md} ·
proyek/docs/_MANIFEST.draft.json
```

Gambar 7.1 — Satu perintah membedah tiga bahan dan mencetak ringkasan ≤15 baris.

1. Per bahan: jumlah entri HAR dan endpoint yang dikenali. Semua bahan dibedah dengan garam hash yang sama, sehingga objek yang sama dikenali lintas peran.
2. Cakupan halaman 73%, dilaporkan apa adanya. Patokan kasar: di bawah 70% sebaiknya tangkap ulang terarah.

3. Temuan berisiko pasif: ada rute yang **mengubah data lewat GET**. Rute seperti ini berbahaya bila di-*prefetch* atau di-*crawl*.
4. Sisa yang belum tertangkap: tiga halaman dan satu formulir. Inilah daftar untuk tangkap ulang terarah.

Keluarannya: `docs/_RECON_WEB/` (dosir dan laporan cakupan) dan `docs/_MANIFEST.draft.json`. Draf ini **tidak pernah menimpa** `_MANIFEST.json` resmi.

> **Bukti tidak bocor.** Setelah demo, folder `docs/` keluaran dicari untuk NIP, nama, dan sandi yang ditanam di *fixture*. Hasilnya nihil.

7.4 Membaca laporan cakupan

Model bekerja dari laporan, bukan dari HAR mentah. Bagian terpenting adalah matriks akses dan transisi status:

docs/_RECON_WEB/laporan-cakupan.md

```
1  `—` tak teramati (bukan dilarang) · `⛔` ditolak nyata.                                  (1)
2
3  | Modul | admin | pegawai |
4  |---|---|---|
5  | cuti | lihat, setujui, tolak | buat, create, hapus, lihat, ⛔403 |                  (2)
6  | dashboard | — | lihat |
7  | notif | — | lihat |
8  | pengaturan | lihat | — |
9
10 ## Entitas teramati (lapisan antarmuka)
11 - **cuti**: id, nip, nama, jenis_cuti, status, tgl_mulai, atasan_id, tgl_selesai, alasan,
   lampiran · relasi: [INFER] atasan_id → atasan · PII: nama, nip
12 - **pengaturan**: kuota_tahunan, mode_persetujuan
13
14 ## Enum / status
15 - `jenis_cuti`: besar=Cuti Besar, sakit=Cuti Sakit, tahunan=Cuti Tahunan
16 - `status`: diajukan, disetujui
17 - `badge`: Diajukan, diajukan
18 - `mode_persetujuan`: berjenjang=Berjenjang, langsung=Langsung
19
20 ## Transisi status
21 - ✅ cuti.status: diajukan → disetujui oleh admin via `POST /cuti/{id}/setujui`          (3)
22 - ❔ kandidat `GET /cuti/{id}/hapus` (pegawai)
23 - ❔ kandidat `POST /cuti/{id}/tolak` (admin)                                           (4)
24
25 ## Alur
```

Gambar 7.2 — Matriks akses per modul, transisi status, dan alur lintas peran.

1. Legenda yang wajib dipahami: `–` berarti **tak teramati**, bukan dilarang. Hanya simbol larangan (lingkaran merah) yang berarti ditolak nyata.
2. Pegawai mendapat 403 nyata saat membuka halaman verifikasi admin. Ini bukti larangan yang sah.
3. Transisi `diajukan → disetujui` teramati sungguhan: objek yang sama berubah status setelah aksi admin.
4. Transisi lain baru berstatus kandidat (bertanda tanya). Agar terbukti, alurnya perlu ditangkap ulang pada objek yang sama.

Sinyal yang perlu Anda periksa di Fase 3:

- **HAR tanpa isi respons**: ekspor ulang dengan opsi "with content".
- **Transisi 0 atau alur lintas peran 0**: peran ditangkap tanpa menjalankan alur utuh pada objek yang sama.
- **Banyak endpoint `[AMATI-JS]`**: fitur tersembunyi di SPA yang belum pernah diklik.

Maksimal dua putaran tambal. Sisanya dicatat jujur sebagai `[TERBUKA]`. Cakupan sempurna bukan syarat; cakupan yang dilaporkan apa adanya adalah syarat.

7.5 Wawancara kilat dan serah ke vcbd

Pertanyaan terpenting di Fase 4 adalah: **tulis ulang atau lanjutkan?** Jawabannya mengubah arti setiap `[AMATI]`. Bila ditulis ulang, perilaku teramati menjadi acuan yang boleh diperbaiki; bila dilanjutkan, ia menjadi kontrak yang harus dijaga. Setelah Ringkasan Temuan dikonfirmasi, koreksi masuk ke `_MANIFEST.draft.json` (dosir adalah buku bukti mesin dan tidak disunting tangan), lalu `vcbd` dimuat dalam mode brownfield. `vcbd` memperlakukan draf itu sebagai fakta diketahui, dan hanya mewawancarai celahnya.

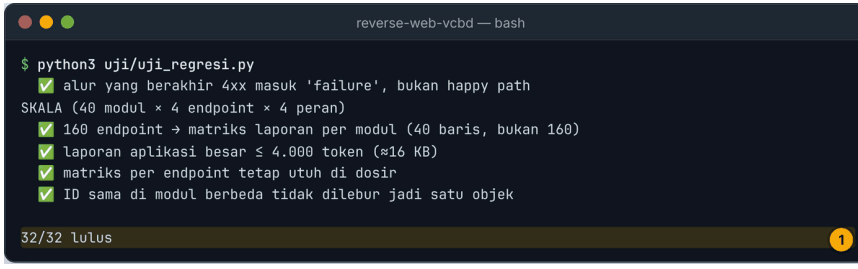
PROMPT

Susun blueprint dari docs/_MANIFEST.draft.json hasil reverse-web-vcbd. Perlakukan sebagai brownfield; wawancarai hanya celahnya.

Rantai yang disarankan: `reverse-web-vcbd` → `vcbd` → `coding-vcbd`.

7.6 Uji regresi skill

Bila Anda mengubah skrip skill ini, misalnya menambah pola PII atau parameter router baru, jalankan uji regresinya. Semua cek wajib lulus.



Gambar 7.3 — Uji regresi `reverse-web-vcbd`: 32 dari 32 cek lulus.

1. Termasuk uji skala: 40 modul × 4 endpoint × 4 peran tetap menghasilkan laporan di bawah 4.000 token.

Latihan 7

1. Tangkap HAR dua peran dari aplikasi instansi yang Anda kelola. Berapa persen cakupan halamannya?
2. Temukan satu sel `–` di matriks akses. Apa yang harus dilakukan untuk mengubahnya menjadi bukti?
3. Jalankan tangkap ulang terarah hanya untuk halaman di daftar "belum".

Rangkuman Bab 7

- Satu HAR per peran, dengan satu alur utuh pada objek yang sama.
- `jalankan.py` adalah cara utama; ringkasannya sering sudah cukup.
- "Tak teramati" bukan larangan; larangan hanya dari 401/403 nyata.
- Skill berhenti di dosir dan draf manifest; `vcbd` yang menulis blueprint.

BAB 8

Merangkai Suite dalam Kerja Sehari-hari

Lima skill sudah kita kenal satu per satu. Bab ini merangkainya menjadi pola kerja: tiga jalur perjalanan proyek, ritme sesi yang hemat, cara bekerja dalam tim, dan cara keluar dari masalah yang paling sering muncul.

8.1 Tiga jalur perjalanan

Titik awal	Rantai skill	Kunci keberhasilan
Ide baru	`vcbd` → `coding-vcbd` → `review-vcbd`	Jangan lompati konfirmasi; slice kecil yang bisa didemokan
Aplikasi warisan berkode	`reverse-vcbd` → `review-vcbd` (PINDAI) → `coding-vcbd`	Izin data read-only; jangan memperbaiki sambil mendokumentasikan
Aplikasi web tanpa kode	`reverse-web-vcbd` → `vcbd` → `coding-vcbd`	Satu alur utuh per objek; putuskan "tulis ulang atau lanjutkan"

Ketiganya bertemu di satu titik: setiap perubahan besar kembali ke **Mode Pembaruan** `vcbd`, supaya dokumen tidak tertinggal dari kode.

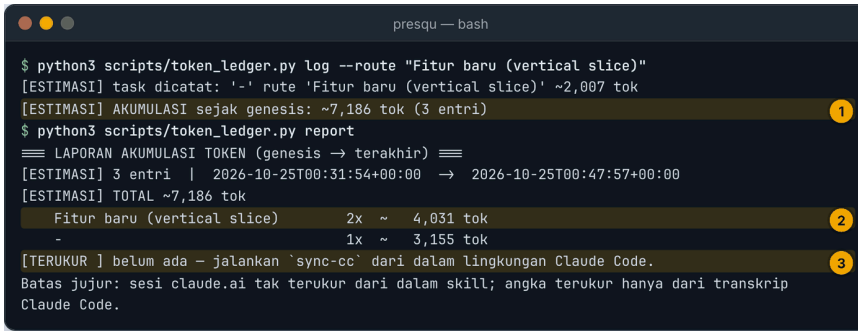
8.2 Ritme satu sesi

Pola yang paling hemat dan paling aman adalah **satu slice per sesi**:

1. Buka sesi baru di root proyek. Prompt pembuka: *"Baca CLAUDE.md lalu INDEX.md, lanjutkan slice berikutnya dari 03_ROADMAP."*
2. Biarkan agen menyampaikan Rencana Slice. Periksa daftar berkasnya, karena di situlah scope dijaga.
3. Ikuti laporan counter di akhir tiap step.
4. Jangan terima "selesai" sebelum `gerbang.sh` berakhir dengan *exit* 0 dan keluarannya ditempel.
5. Tutup slice: arsip 23, status roadmap, satu commit, `fitur-selesai` beserta catatannya.
6. Tutup sesi. Sesi berikutnya mulai dengan konteks bersih.

8.3 Meteran token

Blueprint dirancang agar hemat token, dan meteran membuktikannya. `token_ledger.py` mengakumulasi pemakaian sejak genesis paket:



Gambar 8.1 — Buku besar token PRESQU sejak genesis.

1. Akumulasi sejak scaffold pertama, termasuk catatan dari `coding-vcbd`.
2. Rincian per rute INDEX. Rute yang paling sering dipakai paling layak dioptimalkan.
3. Angka `[TERUKUR]` belum ada. Skrip mengatakannya terus terang dan menunjukkan cara mendapatkannya: `sync-cc` dari dalam Claude Code.

8.4 Bekerja dalam tim

- **Commit blueprint bersama kode.** `docs/`, `CLAUDE.md`, `INDEX.md`, dan `scripts/` adalah bagian repo, bukan catatan pribadi.
- **Jadikan `validate.sh` dan suite tes sebagai cek CI.** Dokumen yang bertentangan sebaiknya ditolak di *pull request*, bukan ditemukan agen berikutnya.
- **Satu pemilik per dokumen kunci.** Misalnya `07` dipegang penanggung jawab basis data, dan `26` dipegang desainer.
- **Review adversarial.** Setelah implementasi, minta subagen dengan konteks segar membandingkan diff terhadap dokumen terkait.
- **Mode SPLIT untuk tim terpisah.** Backend dan frontend yang dikerjakan orang atau sesi berbeda dijembatani `kontrak/openapi.yaml` dan `validate-kontrak.sh`.

8.5 Masalah umum dan jalan keluarnya

Gejala	Penyebab umum	Jalan keluar
`[FAIL] … jalankan skrip dari root proyek`	Skrip gerbang dijalankan dari subfolder	`cd` ke root proyek, lalu ulangi
`[FAIL] 8. Dokumen masih KERANGKA`	Dokumen belum diisi atau penandanya lupa dihapus	Isi dokumen, hapus baris status, atau daftarkan ke `collapsed` bila memang stub
Agen menebak perintah	Ledger perintah belum diisi	`meter.py perintah --set …` dari `11`
`[WARN] Blueprint BERUBAH`	Dokumen bergeser sejak disegel	Pastikan disengaja, catat lewat `--dokumen-diperbarui`, jalankan `validate.sh`

Gejala	Penyebab umum	Jalan keluar
S7 melapor berkas di luar rencana	Perubahan "sekalian lewat"	Jelaskan alasannya, atau kembalikan perubahannya
S8 `[SKIP]` padahal ada UI	Templat UI tidak berekstensi UI yang dikenal	Periksa empat state manual dan catat di `--warn-dilewati`
Temuan mesin terasa salah	`pindai.py` berbasis regex	Verifikasi, lalu `temuan.py tolak --alasan …`
Skill versi lama terpakai	Salinan ganda dari sinkron akun claude.ai	Samakan versi, atau blokir lewat `permissions.deny`
Laporan reverse-web tanpa transisi	Alur tidak ditangkap pada objek yang sama	Tangkap ulang satu alur utuh secara berurutan

8.6 Berkas state yang lahir di proyek

Berkas	Ditulis oleh	Isi
`docs/_MANIFEST.json`	vcbd / reverse-vcbd	State blueprint, pemilik fakta, `referenced_by`, flag
`docs/_MANIFEST.draft.json`	reverse-web-vcbd	Draf manifest dari pengamatan kotak hitam
`docs/_TOKEN_LEDGER.json`	vcbd / coding-vcbd	Akumulasi token per rute
`docs/_CODING_LEDGER.json`	coding-vcbd	Perintah, rencana slice, counter
`docs/_archive/23-*.md`	coding-vcbd	Kriteria fitur yang sudah diterima
`docs/_SERAH_BUILD.json`	coding-vcbd	Kontrak serah-terima untuk auditor
`docs/_TEMUAN.json`	review-vcbd	Dosir Temuan berjenjang

Berkas	Ditulis oleh	Isi
`docs/_RECON/` , `docs/_RECON_WEB/`	reverse-vcbd, reverse-web-vcbd	Buku bukti mesin dan laporan cakupan
`kontrak/openapi.yaml`	vcbd (mode split)	Rumah tunggal endpoint dan payload

8.7 Sepuluh kebiasaan baik

1. Beri nama dan deskripsi yang jelas di prompt pertama.
2. Jawab wawancara dengan menyetujui atau mengoreksi default, bukan mengarang dari nol.
3. Baca Ringkasan Kebutuhan dengan serius; itu satu-satunya gerbang sebelum 28 dokumen ditulis.
4. Terjemahkan selera desain menjadi token, bukan kata sifat.
5. Satu slice, satu sesi, satu commit.
6. Tagih keluaran perintah, bukan klaim.
7. Catat utang yang sengaja diambil saat menutup slice.
8. Audit dimulai dari PINDAI; tambalan dimulai dari tes yang merah.
9. Untuk kode warisan, biarkan data yang memutuskan jalur mana yang mati.
10. Saat fakta berubah, ubah dokumen pemiliknya, lalu jalankan `validate.sh`.

Penutup

Buku ini dimulai dengan dua kegagalan termahal dalam vibe coding: membangun hal yang salah, dan dokumen yang saling bertentangan. Kita menutupnya dengan melihat apa yang sebenarnya diberikan VCBD, dan apa yang tidak.

Yang sudah kita lihat bekerja

Sepanjang buku, setiap klaim ditopang keluaran yang benar-benar dijalankan. `validate.sh` menolak blueprint yang masih kerangka. `gerbang.sh` menolak fitur yang testnya hijau tetapi masih menyimpan `TODO` dan kunci rahasia. `pindai.py` menangkap kueri SQL yang masuk lewat pintu samping, dan tes regresi membuktikan celahnya sebelum ditambal. `db_recon.py` menunjukkan nilai status yang tidak pernah terjadi selama 312 pengajuan. `jalankan.py` menemukan rute yang mengubah data lewat GET hanya dari rekaman peramban.

Bagian yang paling berkesan bagi penulis justru yang tidak direncanakan: bug `database is locked` yang muncul sendiri saat tangkapan layar diambil. Jebakan itu sudah tercatat di `16_DEBUGGING_GUIDE` sejak blueprint disusun. Protokol LACAK membawanya dari gejala ke akar masalah ke tes regresi dalam beberapa langkah yang bisa diulang siapa pun. Itulah inti VCBD: bukan menghilangkan kesalahan, melainkan membuat kesalahan cepat terlihat dan murah diperbaiki.

Yang tidak dijanjikan

VCBD tidak membuat agen AI menjadi tidak pernah salah. Skrip gerbang berbasis heuristik dan regex; kita sudah melihat *false positive* yang harus ditolak dengan alasan, dan syarat S8 yang terlewat karena templat UI berada di berkas Python. Gerbang manual (S1 dan S6) tetap membutuhkan mata manusia. Paket 28 dokumen juga merupakan investasi yang tidak sepadan untuk prototipe sekali buang.

Batas-batas itu ditulis terang-terangan oleh skill-nya sendiri, dan menurut penulis itulah alasan terbaik untuk memercayainya: alat yang mengakui batasnya lebih mudah dipakai dengan benar daripada alat yang mengaku sempurna.

Langkah Anda berikutnya

1. Pilih satu aplikasi kecil yang sungguh Anda butuhkan, dan susun blueprint-nya dengan `vcbd`.
2. Bangun slice pertamanya dengan `coding-vcbd`, dan rasakan bedanya ketika "selesai" berarti *exit* 0.
3. Ambil satu aplikasi warisan di tempat kerja Anda, dan jalankan `reverse-vcbd` dalam mode RECON saja. Laporan cakupannya hampir pasti mengejutkan.
4. Bila menemukan kekurangan pada skill, perbaiki dan bagikan. VCBD adalah perangkat lunak bebas.

Semoga buku ini membantu Anda membangun aplikasi yang benar, dengan cara yang bisa dibuktikan.

Oktober 2026
Syams Ideris

LAMPIRAN A

Peta 28 Dokumen dan Pemilik Fakta

No	Dokumen	Pemilik fakta	Digali dari
—	`CLAUDE.md`	Inti selalu-aktif: prinsip, stack ringkas, perintah tersering, guardrail inti	Ringkasan semua
—	`INDEX.md`	Rute task → dokumen; aturan emas	Ditulis `scaffold.py`
00	EXECUTIVE_SUMMARY	Tujuan dan nilai produk	K1
01	PRD	Fitur dan user story (MVP vs later)	K1, K2
02	SCOPE	Batas in-scope dan out-of-scope	K2
03	ROADMAP	Fase berupa *vertical slice*	K2
04	DOMAIN_MODEL	Istilah dan entitas konseptual	K4
05	USER_ROLE	Peran dan matriks akses	K3
06	BUSINESS_PROCESS	Alur happy path dan alur gagal	K3
07	DATA_MODEL	Skema fisik — **dilarang disalin**	K4
08	ARCHITECTURE	Pola arsitektur dan lapisan	K5
09	STACK	Teknologi, versi, dan yang terlarang	K5
10	DEV_ENV	Prasyarat dan variabel lingkungan	K6
11	COMMANDS	Semua perintah run/test/migrasi/build/deploy	K6
12	PROJECT_STRUCTURE	Pohon folder dan penamaan	K5

No	Dokumen	Pemilik fakta	Digali dari
13	TESTING	Strategi dan cakupan tes	K7
14	ERROR_HANDLING	Pola penanganan galat	K7
15	OBSERVABILITY	Logging dan metrik	K6
16	DEBUGGING_GUIDE	Langkah debug dan jebakan (*landmines*)	Turunan
17	AGENT_WORKFLOW	Alur per-task agen	Turunan
18	REPAIR_RULES	Aturan perbaikan bug	K8
19	TASK_TEMPLATE	Format task	Turunan
20	GUARDRAILS	Larangan operasional	K8
21	SECURITY_RULES	Aturan keamanan	K3, K7
22	CHANGE_POLICY	Git, rollback, operasi irreversibel	K8
23	ACCEPTANCE_CRITERIA	Kriteria per fitur aktif + blok verifikasi	K2, K7
24	DEFINITION_OF_DONE	Definisi selesai universal	K7
25	RELEASE_CHECKLIST	Langkah rilis berurutan + rollback	K8
26	UI_CONVENTIONS *(kondisional)*	Token desain, inventaris, empat state, larangan UI	K9
27	API_CONTRACT *(kondisional)*	Aturan kontrak sisi paket	K10

LAMPIRAN B

Rujukan Cepat Perintah Skrip

vcbd

```
python3 scripts/scaffold.py --root . [--force]       # kerangka +
INDEX + CLAUDE + validator
python3 scripts/scaffold.py --init-kontrak --root .  # mode split:
kerangka kontrak/
bash scripts/validate.sh                             # 11 cek
konsistensi paket
bash kontrak/scripts/validate-kontrak.sh             # gerbang
lintas-paket (split)
python3 scripts/token_ledger.py log --route "<rute>"
python3 scripts/token_ledger.py report | sync-cc | estimate
```

coding-vcbd

```
python3 scripts/meter.py init
python3 scripts/meter.py perintah --set test="…" test_satu="…"
run="…" migrate="…"
python3 scripts/meter.py segel
python3 scripts/meter.py fitur-mulai --fitur "<nama>" --rencana
"a.py,b.py"
python3 scripts/meter.py step-mulai --nama "<lapis>"
python3 scripts/meter.py step-selesai --muat "<berkas dimuat>"
python3 scripts/meter.py fitur-selesai [--deviasi …] [--warn-
dilewati …] [--asumsi …]
        [--dokumen-diperbarui …] [--landmine …] [--ditunda …]
python3 scripts/meter.py lapor | serah
bash scripts/gerbang.sh --fitur "<nama>" [--skip-suite] [--base
<ref>]
```

review-vcbd

```
python3 scripts/pindai.py --root . --keluar docs/_TEMUAN.json [--
ringkas]
python3 scripts/asap.py --basis http://localhost:8000 --dari
docs/_ASAP.json [--temuan docs/_TEMUAN.json]
python3 scripts/temuan.py daftar | lapor
python3 scripts/temuan.py tambah …
python3 scripts/temuan.py setujui --id TEM-001
python3 scripts/temuan.py tolak --id TEM-004 --alasan "…"
python3 scripts/temuan.py tutup --id TEM-001 --bukti "…"
```

reverse-vcbd

```
python3 scripts/recon.py --root <repo> --nama "<Nama>"
python3 scripts/db_recon.py --dosir docs/_RECON/dosir-recon.json
--dari-env <repo>/.env
python3 scripts/db_recon.py --dosir … --dsn "mysql://…" |
"pgsql://…" | "sqlite:///…"
python3 scripts/rakit_manifest.py --dosir docs/_RECON/dosir-
recon.json --root <repo> [--split]
```

reverse-web-vcbd

```
python3 scripts/jalankan.py --nama "<Nama>" --root <proyek> \
        pegawai=pegawai.har admin=admin.har
operator=simpanan/operator/ [--origin host]
python3 uji/uji_regresi.py                          # 32 cek,
wajib lulus semua
```

LAMPIRAN C

Label Bukti dan Status

Label	Dipakai di	Arti
`[ASUMSI]`	vcbd	Dipakai bila tidak dikoreksi pengguna
`[TERBUKA]`	vcbd, reverse-web	Sengaja ditunda
`[ISI:]`	semua	Hanya manusia yang tahu; sengaja dikosongkan
`[USULAN]`	reverse-*	Rekonstruksi penalaran, belum terbukti
`[KODE: path:baris]`	reverse-vcbd	Terbaca di berkas
`[DATA: kueri]`	reverse-vcbd	Terbukti dari agregat basis data
`[AMATI: har:berkas#n]`	reverse-web	Teramati langsung di lalu lintas
`[AMATI-KORELASI]`	reverse-web	Objek sama berubah status lintas peran
`[AMATI-JS]`	reverse-web	Ada di bundle JS, belum pernah dipanggil
`[INFER]`	reverse-web	Disimpulkan dari pola teramati
`[ESTIMASI]` / `[TERUKUR]`	coding, ledger	Token dihitung dari ukuran berkas / diukur dari transkrip
`[PASS]` `[WARN]` `[FAIL]` `[SKIP]`	skrip gerbang	Lulus / perlu pertimbangan / gagal / tidak berlaku

Tingkat temuan review-vcbd

Tingkat	Arti	Batas rilis
KRITIS	Rusak, tidak aman, atau melanggar kriteria terima	Wajib nol

Tingkat	Arti	Batas rilis
TINGGI	Menyimpang dari blueprint pada sumbu berdampak	Wajib nol
SEDANG	Menyimpang tapi terlokalisasi	Boleh ditunda dengan alasan tertulis
RENDAH	Kerapian, utang teknis kecil	Dicatat

LAMPIRAN D

Glosarium

Blueprint
Paket dokumen VCBD (CLAUDE.md, INDEX.md, docs/00–27) yang menjadi sumber kebutuhan bagi agen coding.

Brownfield
Proyek yang mengubah atau melanjutkan sistem yang sudah berjalan.

Collapsed
Dokumen yang sengaja menjadi stub karena isinya sepenuhnya mengikuti default.

Dosir
Buku bukti hasil pemindaian mesin (JSON) yang menjadi bahan blueprint atau audit.

Drift (document drift)
Keadaan ketika dokumen saling bertentangan atau tertinggal dari kode.

Empat state
Kosong, memuat, gagal, sukses: state wajib setiap halaman berdata.

Gerbang keras
Titik berhenti yang menunggu konfirmasi eksplisit manusia sebelum menulis.

Gerbang mesin
Skrip yang menguji syarat dan berakhir dengan *exit code* bukan nol bila gagal.

Greenfield

Proyek yang dibangun dari nol.

HAR

HTTP Archive: rekaman lalu lintas jaringan peramban dalam format JSON.

Landmine (jebakan)

Kejutan pada sistem eksisting yang bisa menyesatkan agen; rumahnya dokumen 16.

Manifest

`docs/_MANIFEST.json`: state bersama blueprint.

Profil native

Proyek tanpa framework; blueprint memikul peran framework.

Rute INDEX

Pemetaan jenis task ke daftar dokumen yang boleh dimuat.

Slice (vertical slice)

Satu fitur yang dibangun utuh dari skema sampai tampilan dan bisa didemokan.

Tier-0

Berkas yang selalu dimuat: CLAUDE.md dan INDEX.md (plus manifest saat kunci sesi).

Token desain

Nilai visual konkret (hex, px, huruf) yang dikunci di dokumen 26.

Vibe coding

Membangun perangkat lunak dengan mendeskripsikan maksud kepada agen AI yang menulis sebagian besar kodenya.

LAMPIRAN E

Kode Sumber Inti Demo PRESQU

Berikut kode logika domain PRESQU setelah semua tambalan di Bab 4 dan Bab 5. Kode lengkap demo berukuran kurang dari 250 baris dan hanya memakai pustaka standar Python.

app/service.py

```
"""Logika pindai QR (docs/06 alur F1)."""
import hashlib
import hmac
import os
from datetime import datetime, timedelta, timezone

from app import repo

WITA = timezone(timedelta(hours=8))
JENDELA = {"masuk": (6, 9), "pulang": (14, 18)}
SECRET = os.environ.get("PRESQU_SECRET", "")


class GalatPindai(Exception):
    def __init__(self, kode, pesan):
        super().__init__(pesan)
        self.kode, self.pesan = kode, pesan


def kode_harian(tanggal, rahasia=SECRET):
    return hmac.new(rahasia.encode(), tanggal.encode(),
hashlib.sha256).hexdigest()[:8]


def pindai(conn, pid, jenis, kode, sekarang=None):
    sekarang = (sekarang or datetime.now(timezone.utc)).astimezone(WITA)
    tanggal = sekarang.strftime("%Y-%m-%d")
    if not hmac.compare_digest(kode, kode_harian(tanggal)):
        raise GalatPindai(422, "Kode tidak berlaku")
    awal, akhir = JENDELA[jenis]
```

```
    if not awal ⩽ sekarang.hour < akhir:
        raise GalatPindai(422, "Di luar jam pindai")
    pegawai = repo.cari_pegawai(conn, pid)
    if not pegawai or pegawai[3] ≠ "pegawai":
        raise GalatPindai(403, "Tidak diizinkan")
    if repo.sudah_hadir(conn, pid, jenis, tanggal):
        raise GalatPindai(409, "Sudah tercatat")
    repo.catat_hadir(conn, pid, jenis, sekarang.isoformat())
    return {"status": "tercatat", "jenis": jenis, "waktu":
sekarang.strftime("%H:%M")}
```

app/repo.py

```
"""Satu-satunya pintu SQL (docs/12)."""


def cari_pegawai(conn, pid):
    return conn.execute("SELECT id, nip, name, role FROM employees WHERE
id = ?", (pid,)).fetchone()


def catat_hadir(conn, pid, jenis, waktu):
    conn.execute("INSERT INTO attendances (employee_id, type,
scanned_at) VALUES (?, ?, ?)",
                 (pid, jenis, waktu))
    conn.commit()


def sudah_hadir(conn, pid, jenis, tanggal):
    q = "SELECT 1 FROM attendances WHERE employee_id = ? AND type = ?
AND substr(scanned_at,1,10) = ?"
    return conn.execute(q, (pid, jenis, tanggal)).fetchone() is not None


def cari_nama(conn, nama):
    return conn.execute("SELECT id, name FROM employees WHERE name LIKE
?", (f"%{nama}%",)).fetchall()
```

LAMPIRAN F

GNU General Public License versi 3

Teks buku ini dilisensikan di bawah GNU General Public License versi 3. Teks lisensi resmi berbahasa Inggris dicantumkan utuh di bawah ini, sebagaimana disyaratkan lisensi tersebut.

GNU GENERAL PUBLIC LICENSE
Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the

earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but

which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of

technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family,

or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by

this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions;

the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever

licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that

country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY
GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms,

reviewing courts shall apply local law that most closely approximates
an absolute waiver of all civil liability in connection with the
Program, unless a warranty or assumption of liability accompanies a
copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest
possible use to the public, the best way to achieve this is to make it
free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest
to attach them to the start of each source file to most effectively
state the exclusion of warranty; and each file should have at least
the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see <https://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short
notice like this when it starts in an interactive mode:

<program> Copyright (C) <year> <name of author>
This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate
parts of the General Public License. Of course, your program's commands
might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school,
if any, to sign a "copyright disclaimer" for the program, if necessary.
For more information on this, and how to apply and follow the GNU GPL, see
<https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/licenses/why-not-lgpl.html>.